

OSINT Collection from Social Media Network

B-Tech-AI/DS, Semester-VI

Prepared at



Bhaskaracharya National Institute for Space Applications & Geo-informatics
Ministry of Electronics and Information Technology, Govt. of India.
Gandhinagar

Prepared By

Sayantan Mandal

Shivam Kumar

Rishav Pal

ID No. 26BISAG0507

ID No. 26BISAG0508

ID No. 26BISAG0509

Guided By:

Dr. Shweta Saharan

Department of AI&DS

GSV, Vadodara.

External Guide:

Dr. Jhummarwala Abdul

Additional Director,

BISAG-N, Gandhinagar

SUBMITTED TO

Gati Shakti Vishwavidyalaya



Gati Shakti Vishwavidyalaya - Vadodara

**Bhaskaracharya National Institute for Space Applications
and Geo-informatics**

MeitY, Government of India

**प्रमाणपत्र
CERTIFICATE**

दिनांक/Date: _____

यह प्रमाणित किया जाता है कि गति शक्ति विश्वविद्यालय, वडोदरा के छठे सेमेस्टर बी.टेक-एआई
एंड डीएस के छात्र श्री सयंतन मंडल, श्री शिवम कुमार और श्री ऋषव पाल ने बायसेग-एन में अपनी छठे
सेमेस्टर की ग्रीष्मकालीन इंटरनीशिप सफलतापूर्वक पूरी कर ली है। उन्होंने 18 मई 2026 से 13 जुलाई 2026 तक
सोशल मीडिया नेटवर्क से ओएसआईएनटी संग्रह हेतु वेब-आधारित एप्लिकेशन विकास पर कार्य किया है।

जहाँ तक हमें सूचना है, यह उनके द्वारा किया गया एक मौलिक और प्रामाणिक कार्य है। संस्थान में अपने
कार्यकाल के दौरान, उन्होंने आत्म-प्रेरणा और कर्तव्यनिष्ठा का परिचय देते हुए काम किया। हम उनके उज्ज्वल
भविष्य की कामना करते हैं।

This is to certify Mr. Sayantan Mandal, Mr. Shivam Kumar and Mr. Rishav Pal, students of 6th Semester B-Tech-AI/DS from Gati Shakti Vishwavidyalaya, Vadodara, have satisfactorily completed the 6th semester summer internship at BISAG-N. They have worked on Web-based application development for OSINT Collection from Social Media Network, from May 18th, 2026 to July 13th, 2026.

To the best of our information this is an original and bonafide work done by them. During their tenure at the institute, they demonstrated self-motivation and conscientiousness. We wish them a successful future ahead.

डॉ. झुम्मरवाला अब्दुल
Dr. Jhummarwala Abdul
अपर निदेशक
Additional Director,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

पुनीत लालवानी
Punit Lalwani
अपर निदेशक सह सीआईएसओ
Additional Director cum CISO,
बायसेग-एन, गांधीनगर, गुजरात
BISAG- N, Gandhinagar, Gujarat

About BISAG-N



Modern day planning for inclusive development and growth calls for transparent, efficient, effective, responsive and low-cost decision-making systems involving multi-disciplinary information such that it not only encourages people's participation, ensuring equitable development but also takes into account the sustainability of natural resources.

The applications of space technology and Geo-informatics have contributed significantly towards the socio-economic development. Taking cognizance of the need of geo-spatial information for developmental planning and management of resources, the department of Ministry of Electronics and Information Technology, Government of India, established "Bhaskaracharya National Institute for Space Applications and Geo-informatics" (BISAG-N).

BISAG-N is ISO 9001, ISO 27001, ISO 19115, ISO 19157 and CMMI: L5 certified national institute. BISAG-N, which was initially set up to carry out space technology applications, has evolved into a center of excellence, where research and innovations are combined with the requirements of users and thus acts as a value-added service provider, a technology developer and as a facilitator for providing direct benefits of space technologies to the grass root level functions/functionaries.



BISAG- N Journey

The journey of Institute started as a state government organization, named as **Remote Sensing and Communication Centre (RESECO)** operating under a limited mandate from 1997 to 2003, focusing on utilization of space technologies such as satellite communication, remote sensing and geospatial applications for supporting various developmental initiatives exclusive for the State of Gujarat. Opinions of various stakeholders at grass root levels was obtained to ensure alignment of Institute's work with the development and welfare need of the society. Numerous sessions of different strata of society were organized including Member of Legislative Assembly (MLAs), village sarpanch, farmers, various skill workers, women and students to obtain insights into what are the needs with simplicity of solutions, and processes of their groups and how technology can fulfil those needs and alleviate any constraints for improving the quality of life.

The basic works of maps generation as per the needs of various state government departments laid the groundwork for the institute's advancement. In December 2003, the name of the institute changed as **BISAG (Bhaskaracharya Institute for Space Applications and Geo-informatics)** with an enhanced mandate to promote satellite communication-based education and skill development facility thereby ensuring increased and more actively involvement in governance processes. The name of the institute was changed to honor the celebrated 12th-century mathematician and astronomer, Bhaskaracharya (1114-1185 AD) who provided in-depth insights about various astronomical and mathematical concepts, formulae and calculations.

The institute's services were extensively utilized by various departments of Gujarat Government under the leadership and patronage of the then Honorable Chief Minister Shri Narendrabhai Modi. Numerous times the assembly meetings were held in BISAG-N premises to amalgamate governance and technology for the well-being of the state. The state leadership expressed to the ministers and bureaucrats on multiple occasions the importance of visualization of development plans and schemes on maps and thereby harnessing the capabilities of BISAG as a **"go-to place"** for various technological and geo-informatics solutions required by concerned ministries and departments for the benefit of citizens and development of the state.

On one of such occasions in 2004, Honorable Chief Minister Shri Narendra Modi said, "Friends, I can clearly visualize that within a span of five years, all streams of government administration shall converge into BISAG. This institute shall be a huge treasure of information, if any decisions are to be taken or any department require any information or need any policy change; the **'Source of Information'** shall be BISAG. There for earlier we can transform BISAG into a **'Source of Information'** and position it as back-bone of all governance and administration activities, and the sooner the benefits of science and technology shall be available to the society."



Since then, inspired by above and gaining insights while working collaboratively for various development initiatives of central and state government department, the Institute is focused on following vision:

"To empower and serve the nation through development of space and geo-spatial technologies, its applications, solutions and services in accordance to governance principles for socio-economic welfare of the society."

The Institute develops geo-spatial technologies, databases, software and processes along with skilled resources to provide visualization, analysis and decision support system to various state government

departments. Also, the Satellite Communication infrastructure was enhanced to add more channels and developed in house expertise to install, operate and maintain various electronic instruments and equipment. The institute constantly ensured adoption of national and international standards as well as best practices to develop and reinforce the solutions delivery process. With the improved confidence and trust of the state government to utilize geo-spatial and satellite communication services for developmental, social welfare and disaster management areas, the institute was supported by the Government of Gujarat to construct dedicated buildings for ensuring services are provided at even larger scale. This expansion broadened the institute's scope to integrate advanced techniques such as managing large databases, mathematical modelling, enabling prediction and simplified architecture for the solution aligning with the government priorities for improving efficiency, transparency and pace of development

Along with providing the e-governance support to Gujarat State, the institute also started supporting various other states and central government ministries as and when required for map and decision support system development services. Recognizing this potential, the institute's mandate was further enhanced to cater to the geo-spatial and satellite communication-based e-governance services to all central government ministries as well as various states and union territories as a **National** institute. The name of the institute was then changed as Bhaskaracharya **National** Institute for Space Applications and Geo-informatics (**BISAG-N**), registered as an Autonomous Scientific Society, as per the Societies Registration Act, 1860, under the Ministry of Electronics and Information Technology (MeitY), Government of India (GoI). BISAG-N; enhanced mandate includes to undertake technology development & management; research & development; facilitate national and international cooperation and capacity building; support technology transfer and entrepreneurship development in area of geo-spatial technology.

Today, BISAG-N is collaboratively working with 50+ central government ministries and most of the states and union territories for supporting various developmental programs and citizen centric services. BISAG-N provides the technological support to these stakeholders from its Gandhinagar and New Delhi facilities. These facilities have adequate skilled personnel and state-of-the-art infrastructure for providing geo-spatial solutions and software applications including support of satellite communication-based education and training. Thus, BISAG-N is playing pivotal role as a one stop center for providing geo-spatial and space technologies for various flagship programme being undertaken for the development of nation in line with geo-spatial education, logistics and various other policies and program of central and state governments.



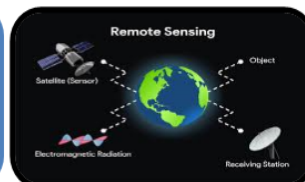


Satellite Communication:

For promotion and facilitation of the use of broadcasting facility for distant interactive training, education, skill development, health, agriculture and extension

Remote Sensing:

For inventory, mapping, developmental planning and monitoring of natural and built resources

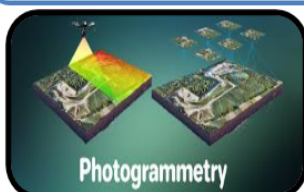


Geographic Information System:

For conceptualization, creation and organization of multi-purpose common digital databases for development of sectoral/integrated decision systems.

Global Navigation Satellite System:

For location based services, engineering applications and research

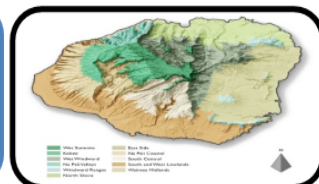


Photogrammetry:

For creation of maps, Digital Elevation Model (DEM), terrain characteristic maps for resource planning and management

Cartography:

For Thematic Mapping and generation of value added maps



Software Applications Development:

For wider usage in geo-spatial applications, Decision Support Systems (DSS) and ERP solutions.

Innovation, Research & Development:

Quantum Computing, Digital Twins, Cyber Security, Block Chain, Big Data, AI/ML

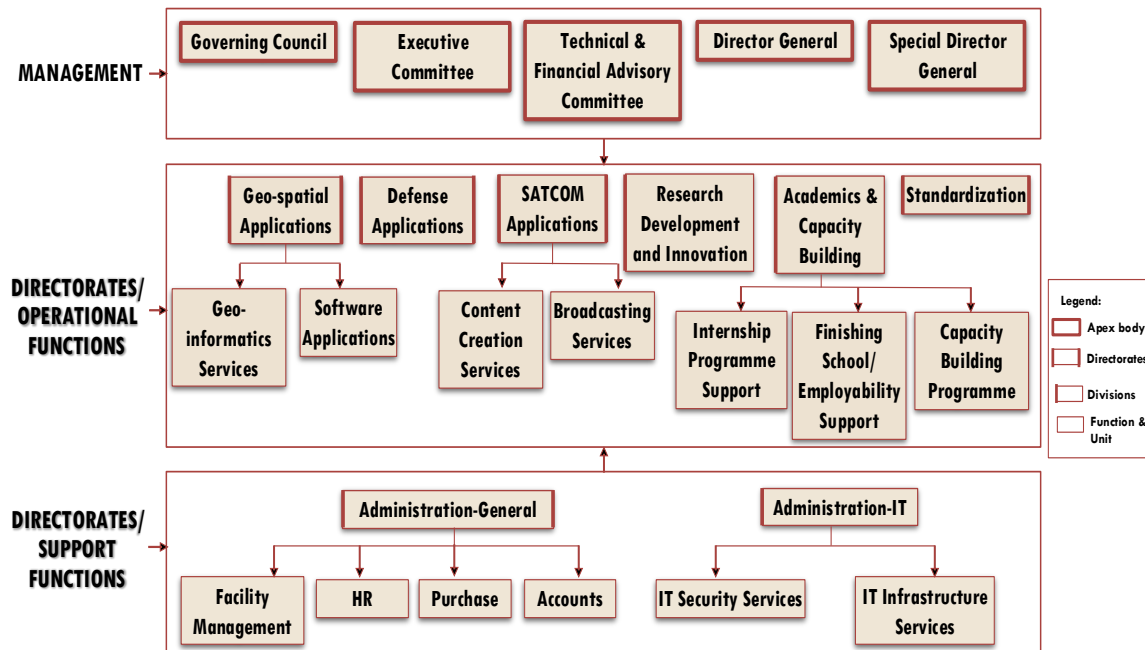


Education and Training:

For providing Education, Training & Technology knowledge transfer to large number of students, end users and collaborators for promoting Research & Development

Organizational Setup

BISAG-N main domain areas include Satellite Communication, Geo-spatial Solutions and Technology Development, Geo-informatics Applications Development, Standardization, Academics & National/ International Co-operation and Administration.



BISAG-N: Functions and Divisions

SATCOM Applications

The SATCOM Directorate provides services of distance education, capacity building, training, extension, awareness and empowerment using the state-of-the-art facility comprises of an uplink earth station, control room, studios and a satellite communication-based network. The services are primarily utilized by Ministry of Education and various other central and states ministries. Through Ministry of External Affairs (MoEA), BISAG-N is now poised to provide SATCOM based education to neighbouring countries also.

SATCOM operations are inspired by the wisdom of Hon'ble Prime Minister of India as elicited on numerous occasions. For example, in 2004 as Hon'ble Chief Minister of Gujarat, Shri Modi explained the importance of learning through the ancient way of visualization using virtual media with these words:

“The way Eklavya studied by making a statue, why we can't study with the help of a T.V.? Yes, we can. Was any cinema song ever taught to us in any class? Was it ever taught by any teacher? We learnt song from T.V., isn't it? If we have learnt entire cinema song, learnt every word, learnt scale, learnt rhythm, it means we can learn through T.V. also.”



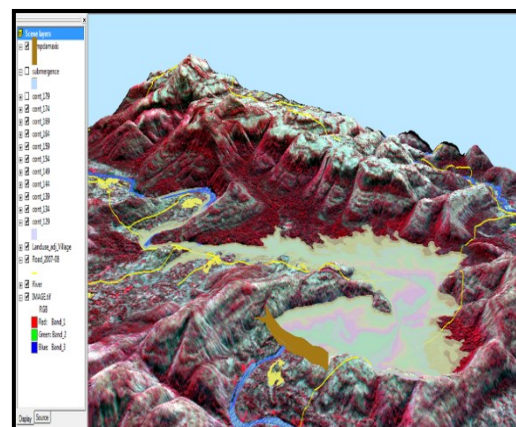
There are twelve state-of-the-art studios for live recording of content supported by dedicated editing labs, subtitling, translation, animation and various other aspects. The institute has set up ten studios at Gandhinagar facility and another two studios at New Delhi. Infrastructure for transmission and Ku band uplink teleports (11m/9.3m) of more than 350 channels is installed, integrated and maintained by inhouse team. BISAG-N support various initiatives of Ministry of Education, Government of India, as well as other central and state government ministries such as PM e-Vidya, Swayam Prabha etc. as detailed further.



Geo-spatial Applications Directorate

BISAG-N's Geo-Spatial Applications Directorate provides specialized services and solutions to implement Geographic Information Systems (GIS) for providing impetus to planning and developmental activities at grass root level. The objective of GA directorate is to provide analysis and decision support services to various stakeholders through scientifically organized, comprehensive, multi-purpose, compatible and large scale geo-spatial databases.

Geo-spatial Applications Directorate creates and organizes large-scale and multi-purpose geospatial datasets in collaboration with central and state government departments and agencies. A well-qualified team of GIS technicians and engineers ensure that the datasets are compatible, standardized and authenticated for use in analysis, planning and decision-support functions. Goals and roles of converging organizations are taken into consideration while geo-spatial database creation and software development activities



Software Development Division

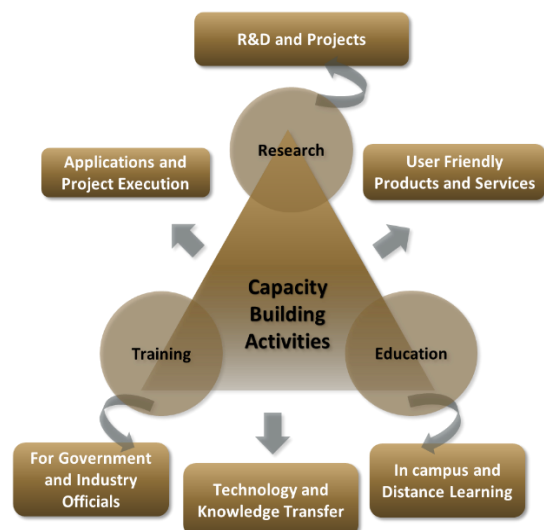
The geo-spatial solutions are packaged and provided as software products for ease of retrieval of spatial and feature attributes data, enabling wide range of analysis, prediction and decision support services. Software technologies like Java, Python, JavaScript, Spring Boot, React JS, Node JS, Microservices, XML, GeoJSON, GeoServer, QGIS, Android and iOS frameworks are used to develop user-friendly and reliable solutions.



Academy of Geo-informatics

BISAG-N, being a national institute for geo-spatial and emerging technology-based applications, has a strong focus on research, development, capacity building and employability support for under-graduate, post-graduate and doctoral students. The institute's infrastructure and supportive work environment provide an ideal platform for students pursuing thesis work in a variety of technologies and domains.

BISAG-N is equipped with cutting-edge infrastructure to support research across multiple disciplines. Dedicated laboratories for research on the application of artificial intelligence, machine learning, GIS (Geographic Information Systems), remote sensing, quantum computing, IoT (Internet of Things) and other emerging technologies are established high-performance computing facilities for intensive data processing tasks are available in the institute. Open-source platforms and in-house developed software are available for projects, research and innovation activities. More than 300 students are supported every year for learning practical based aspects of geo-spatial and emerging technology applications while completing their projects.



BISAG-N Finishing School Programme (FSP) supports wards of special groups of society such as scheduled tribe, scheduled caste, non-gazetted security, armed forces citizens and martyr etc, for development of technical and soft skills to improve their employability including support for entrepreneurship and employment opportunities.

The school provides basic and specialized training to help participants develop capacities as required by the for related industry/domain. The participants undergo thorough counselling sessions as well as mock/actual interviews for improving employability in relevant industries as per the interest of the candidates. Along with the skills training the participants are also allowed to observe engaged in ongoing projects of organization. BISAG-N academic activities also provide capacity building support to government officials for effectively using geo-spatial solutions and decision support systems for performing planning and monitoring for various government schemes and initiatives.

Industrial visits of school students are organized to provide them exposure of space applications and geo-informatics thereby supporting the development of scientific temperament at early stage. More than 5000 students visit the premises every year to get exposure of the activities being carried out at BISAG-N.



Standardization Directorate

BISAG-N has a dedicated directorate for ensuring continuously focus on innovation, standardization and institutionalization of solutions & service. The Standardization Directorate focuses on implementation of various national and international standards related to program solutions and data management. The directorate develops internal SOPs and policies for compliance to these standards and provides training for effective implementation. The directorate also liaison with the various departments and external agencies during internal and external audits. In addition, the directorate acts as member in standards committees of Bureau of Indian Standards (BIS) and other standard developing bodies for development of standards.

Goals of Standardization Directorate include:

- To proactively implement product and process improvements initiatives using various national & international standards and best practices. This includes ensuring quality and reliability of geo-spatial data, completeness, updation of metadata, functional performance and security reviews of applications, capacity building support to end-users for adoption of solutions and information security-oriented culture development
- To develop a culture of efficiently working as per standards while fostering innovation
- To perform reviews, gap analysis, training, documentation and standards implementation support activities

Administration Directorate

Administrative Directorate provides strategic and management support to various activities of BISAG-N. The directorate ensures infrastructure and facilities management in addition to providing services such as coordination during visits of various delegations, logistic support to BISAG-N staff during travel, accounting and finance, procurement, training, safety and security, legal compliance, maintenance, housekeeping, horticulture, construction and extension projects, campus monitoring, canteen management as well as celebrations and welfare activities. The directorate also ensure recruitment and human resource development as per the needs of various projects and directorates while upholding principles of equality, fairness and diversity in work force.

BISAG-N has adequate human resources, well trained for behavioral, functional and domain competencies, with clear roles and responsibilities in alignment with service delivery expectations. The staff well-being and motivation initiatives are ensured for maintaining a culture of shouldering responsibilities proactively and delivering high-performance utilizing multi-disciplinary knowledge.

Administration Directorate ensure strategic and compliance support to operations of the institute by managing human resources, facilities, expansion projects, finance and governance aspects. The directorate also coordinates with Council stakeholders and members of Governing and Executive committee as well as other high-level committees.

Defence Applications Directorate

This directorate develops space, geo-spatial, AI and other emerging technologies-based applications under the guidance of various agencies of the Ministry of Defence, Government of India. All the technologies used are developed inhouse of customized open source. Considering the strategic significance of these projects, dedicated team and premises are allocated for ensuring information security. Most of the critical activities and data management are performed directly at various defence establishments.

Besides the projects of various infrastructure and social welfare ministries and departments of central and state government, BISAG-N has a dedicated directorate for supporting defence and security departments for providing data products, decision support systems and services involving geo-spatial and various other emerging technologies. The projects are executed in high security dedicated laboratory for various units of army, navy and air-force for supporting strategic and operational aspects of the operations.

Through its innovative geospatial solutions and technological expertise, BISAG-N is playing a crucial role in enhancing the nation's defence capabilities and ensuring long-term strategic autonomy. Due to nature of high secrecy of these strategic projects, they are not included in details in this compilation.

Research and Development

BISAG-N's research and development group supports all other directorates for providing solutions related to new definitions and technology. The group develops innovative applications, features, tools and services utilizing technologies such as remote sensing, photogrammetry, artificial intelligence, machine learning, internet of things, computer vision, big data and quantum computing, which are later integrated in live projects.

BISAG-N has made significant progress for the development of Artificial Intelligence/ Machine Learning (AI/ML) applications. Algorithms and models have been developed which can be customized and used with training, validation and live data upon their data for accurate predictions and smarter results.

The applications developed by the R&D team include support for the pluggable APIs which can use the machine learning with minimal code reconfiguration. Various GIS applications are developed at BISAG-N. Utilizing spatial and non-spatial processing of Big Data and Web GIS architecture These applications support multi-terabyte datasets and large number of consumers. Big Data applications for visualization developed by the institute has enabled several projects to explore and analyse multiple large archives. Through continuous research and development, BISAG-N's R&D capabilities are continuously enhanced. This allows to enhance users' satisfaction, growth and success stories.

Cyber Security Cell

Cyber Cell reviews and update the threat landscape for the BISAG-N's critical infrastructure on a regular basis in line with various vulnerabilities detected and published by reputed repositories. The cell verifies the applications using Vulnerability Assessment & Penetration Testing (VAPT) ensuring the compliance to Guidelines for Indian Government Websites (GIGW), Open Web Application Security Project (OWASP), SANS 25 etc. The cell also ensures implementation of good information security practices in line with ISO 27001 such as serious logs review, vulnerability analysis, exception reporting, maintenance of firewall and other network monitoring tools.

CANDIDATE’S DECLARATION

We declare that the 6th semester internship project report entitled “**OSINT Collection From Social Media Network**” is our own work conducted under the supervision of the external guide **Dr. Jhummarwala Abdul** from BISAG-N ([Bhaskaracharya National Institute for Space Applications & Geo-informatics](#)). We further declare that to the best of our knowledge, the report for this project does not contain any part of the work which has been submitted previously for such project either in this or any other institutions without proper citation.

Candidate 1’s Signature

Sayantan Mandal

Student ID: 26BISAG0507

Candidate 2’s Signature

Shivam Kumar

Student ID: 26BISAG0508

Candidate 2’s Signature

Rishav Pal

Student ID: 26BISAG0509

Submitted To:

Gati Shakti Vishwavidyalaya – Vadodara.

ACKNOWLEDGMENT

We are grateful to **Shri T.P. Singh**, Director General (BISAG-N), for giving us this opportunity to work the guidance of renowned people of the field of GIS-Based Portal, also providing us with the required resources in the company.

We would like to express our endless thanks to our external guide, **Dr. Jhummarwala Abdul**, and to the Training Cell, **Mr. Punit Lalwani & Mr. Sidhdharth Patel** at Bhaskaracharya National Institute of Space Application and Geo-informatics, for their sincere and dedicated guidance throughout the project development.

Also, our hearty gratitude to our Head of Department, **Dr. Vipul Kumar Mishra** and our internal guide **Dr. Shweta Saharan** for giving us encouragement and technical support on the project.

Sayantana Mandal.

Student ID: 26BISAG0507

Shivam Kumar.

Student ID: 26BISAG0508

Rishav Pal.

Student ID: 26BISAG0509

ABSTRACT

The rapid expansion of public discussions and statements on social network like X (formerly Twitter) has created both a major opportunity and a challenge for researchers. It is an opportunity because vast amounts of public data can be analyzed to understand how narratives spread, how networks of influence are built, and how coordinated communication occurs. It is a challenge because this data is unstructured, massive in size, and difficult to study manually. To address this, our project introduces a system designed to gather and analyze open-source data. The use of Open-Source Intelligence (OSINT) is central to this project, as it refers to the collection and evaluation of publicly accessible information to generate meaningful insights. Since the system relies strictly on public posts, hashtags, and profile details that are visible to anyone on the web—without violating privacy settings or accessing private accounts—it operates entirely as a core OSINT methodology. The project automates data collection from a selected set of prominent Indian political and media accounts using browser automation tools. It then extracts key elements—such as hashtags, user mentions, external web links, and categories—and stores them in two database systems: a relational database (PostgreSQL) for structured tables and a graph database (Neo4j) to map connections and relationships.

Developed in a step-by-step approach, our project transitioned from a basic command-line tool into a secure web application. It features a backend server and a multi-page web dashboard for visual analysis. The system includes an interactive network explorer and an automated report generator that outputs findings directly into standard PDF format, helping users easily digest relationship patterns and coordinated campaigns in plain language. During testing, the system successfully identified several real-world open-source intelligence patterns, such as coordinated hashtag usage, unusual mention behaviors, and cross-promotional connections among media handles. This report presents the requirements, system architecture, database design, testing processes, and evaluation of the project, concluding with future improvements and system limitations.

CONTENTS

Chapter No.	Description	Page No.
-	List of Figures	I
-	List of Tables	II
-	List of Abbreviations	III
-	List of Definitions	IV
-	List of Screenshots	V
1	Introduction	1
1.1	Project Details	2
1.2	Purpose	3
1.3	Scope	4
1.4	Objectives	5
1.5	Technology Stack Used	5
1.6	Literature Review	7
2	Project Management	9
2.1	Feasibility Study	9
2.1.1	Technical Feasibility	9
2.1.2	Time Schedule Feasibility	10
2.1.3	Operational Feasibility	11
2.1.4	Implementation Feasibility	11
2.2	Project Planning	12
2.2.1	Development Approach & Justification	12
2.2.2	Milestones & Deliverables	13
2.2.3	Roles & Responsibilities	14
2.2.4	Group Dependencies	14
2.3	Project Scheduling (Gantt/PERT chart)	15
3	System Requirement Study	17
3.1	Existing System Overview	17
3.2	Limitations of the Existing System	17
3.3	User Characteristics	18
3.4	Functional Requirements	19
3.5	Non-Functional Requirements	20
3.6	Hardware & Software Requirements	21
3.7	Constraints	22
3.7.1	UI Constraints	23
3.7.2	Communication Interface	23
3.7.3	Hardware Interface	23
3.7.4	Criticality of Application	24
3.7.5	Safety & Security Considerations	24
3.8	Assumptions & Dependencies	24
4	Proposed System Requirements	26
4.1	Overview of Proposed System	26
4.2	Module Descriptions	27
4.3	System Features	28

Chapter No.	Description	Page No.
4.4	Advantages of Proposed System	29
5	System Design	30
5.1	System Architecture Design	30
5.2	UML Diagrams	33
5.2.1	Use Case Diagram	33
5.2.2	Activity Diagram	35
5.2.3	Sequence Diagram	37
5.2.4	Class Diagram	39
5.2.5	Entity-Relationship (E-R) Diagram	41
5.2.6	DFD Diagram	43
5.3	Database Design	44
5.3.1	Table Design & Relationships	44
5.3.2	Normalization	46
5.3.3	Data Dictionary	47
5.4	GUI Design	48
5.5	Screenshots	49
6	Implementation Planning	56
6.1	Implementation Environment	57
6.2	Tools & Technologies Used	58
6.3	Coding Standards Followed	58
7	Application	60
7.1	Narrative and Trend Monitoring	60
7.2	Coordinated Activity Detection	60
7.3	Narrative and Influence Analysis	60
7.4	Intelligence Reporting and Sharing	61
8	Limitations and Future Scope	62
9	Conclusion and References	64
9.1	Conclusion	64
9.2	References	65
10	Appendices	66
11	Report Verification Procedure	68

List of Figures

Figure No.	Figure Description	Page No.
1	Project Gantt Chart	16
2	High-Level Design (HLD) of the Linkmap OSINT Platform	31
3	Low-Level Design (LLD) of the Linkmap OSINT Platform	32
4	Use Case Diagram	34
5	Activity Diagram	36
6	Sequence Diagram	38
7	Class Diagram	40
8	Entity- Relationship (E-R) Diagram	42
9	DFD Level 0 — Context Diagram	43
10	DFD Level 1 – Module Decomposition	43
11	Postgres Schema	45
12	Neo4j Graph Model Schema	46

List of Tables

Table No.	Table Description	Page No.
1	Project Fact Sheet	2
2	Technology Stack Summary	6
3	Weekly Milestones & Deliverables	13
4	Roles & Responsibilities	14
5	Inter-Role Dependency Matrix	15
6	Primary vs Secondary User Characteristics	19
7	Hardware & Software Requirements	22
8	Summary of Delivered System Features	28
9	PostgreSQL Table Design	44
10	Neo4j Node & Relationship Design	45
11	PostgreSQL Data Dictionary (profiles & tweets)	47
12	Summary of Limitations and Recommended Future Work	63
13	Configuration File Reference	66
14	Representative API Endpoint List	66

List of Abbreviations

Abbreviation	Full Form
OSINT	Open-Source Intelligence
API	Application Programming Interface
REST	Representational State Transfer
WS	WebSocket
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
JSONB	Binary JSON (PostgreSQL data type)
CSV	Comma-Separated Values
PDF	Portable Document Format
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
CORS	Cross-Origin Resource Sharing
UI/UX	User Interface / User Experience
SPA	Single-Page Application
DOM	Document Object Model
ES	ECMAScript
UML	Unified Modeling Language
E-R/ERD	Entity-Relationship (Diagram)
DFD	Data Flow Diagram
UAT	User Acceptance Testing
TTL	Time To Live
RK/FK	Primary Key / Foreign Key
CDN	Content Delivery Network
ASGI	Content Delivery Network
CLI	Command-Line Interface
IP	Internet Protocol
ReDoS	Regular Expression Denial of Service
HLD/LLD	High-Level Design / Low-Level Design
FR/NFR	Functional Requirement / Non-Functional Requirement

List of Definitions

Term	Definition
OSINT Lifecycle	The standard intelligence workflow of planning, collection, processing, analysis and dissemination.
Property Graph	A graph data model (used by Neo4j) in which both nodes and relationships can hold key-value properties.
Idempotent Operation	An operation that produces the same end result no matter how many times it is repeated (e.g. an upsert).
Rate Limiting	Restricting the frequency of requests or actions to avoid overloading a system or triggering anti-bot detection.
Deduplication	The process of detecting and skipping records that have already been collected in a previous run.
Connection Pool	A cache of pre-established database connections reused across requests instead of opening a new one each time.
Force-Directed Graph	A graph visualisation layout that positions nodes using simulated physical forces (attraction/repulsion) so connected nodes cluster naturally.
Skeleton Loader	A placeholder UI animation shown while real data is still being fetched, in place of a blank screen or spinner.
Selector (Web Scraping)	A CSS/DOM query pattern used to locate a specific element on a web page for data extraction.
Stealth Scraping	Browser-automation techniques that disguise automated activity as ordinary human browsing to avoid bot detection.
Graceful Degradation	A design Principle where a system continues operating with reduced functionality when one component fails, rather than failing entirely.
Whitelist	An explicit, fixed list of permitted values used to validate input safely (e.g. allowed sort columns).
Burst Detection	Identifying an unusually high, time-concentrated spike in an activity (e.g. hashtag usage) that suggests a coordinated event.

List of Screenshots

Screenshot No.	Screenshot Description	Page No.
1	Overview — landing page with live stat counters and animated network preview	49
2	Handle Intelligence — filterable handle directory with cluster bars	50
3	Influence Network — D3.js force-directed graph with Graph Search panel	50
4	Hashtag Analysis — frequency bars, co-occurrence table, burst alert	51
5	Topic Clusters — expandable topic rows with distribution chart	51
6	Campaign Detection — coordinated-activity alert and radial hub	52
7	Tweet Analysis — six-filter search table with SVG charts	52
8	Scraper Control Panel — live WebSocket progress feed and job history	53
9	Data Explorer — seven-filter PostgreSQL search with CSV export	53
10	Intelligence — bridge nodes, chains, clusters and anomaly flags	54
11	Nexus Graph Explorer — split-panel graph with autocomplete and side panel	54
12	Automated PDF Report — sample generated intelligence report	55

1 Introduction

Social networks have become the primary arenas in which political narratives, media campaigns, and public opinions are formed, shared, and discussed. In India, public social networks like X (formerly Twitter) host a dense, highly interconnected ecosystem of political leaders, television news channels, journalists, fact-checkers, and civil-society accounts. The posting behaviour, hashtag usage, and mention patterns on these social networks carry substantial analytical value for anyone trying to understand how narratives spread, who amplifies whom, and where coordinated activity may be occurring. Investigating these social networks manually—one account, one hashtag, or one mention at a time—is slow, error-prone, and cannot scale beyond a handful of accounts observed over a very short window of time.

This chapter introduces the internship project undertaken at the Bhaskaracharya National Institute for Space Applications and Geo-informatics (BISAG-N): the design and development of an automated Open-Source Intelligence (OSINT) collection and analysis platform for public social networks. OSINT represents the practice of collecting and analyzing information from public sources to generate useful findings. In this project, public social networks serve as the primary source, and this system helps automate the collection and mapping of this data. The chapter begins by describing the project in concrete terms—who supported it, who built it, and over what period (Section 1.1)—before explaining why the project was undertaken (Section 1.2, Purpose) and defining what was and was not attempted within the internship timeframe (Section 1.3, Scope). Section 1.4 lists the specific technical objectives we set at the outset. Section 1.5 summarizes the technology stack chosen to meet those objectives, and Section 1.6 places this project within the broader landscape of OSINT methodology, graph database theory, and existing social network analysis tools. This section explains where this system sits relative to existing tools and why a custom, integrated system was preferred over assembling separate third-party solutions.

More broadly, this report is organized to mirror the natural lifecycle of a data platform project. Chapters 2 and 3 cover project management and requirement analysis respectively, establishing how the work was planned and what the system needed to do. Chapter 4 proposes the system that was ultimately built, and Chapter 5 details its architecture, database design, and user interface layouts. Chapters 6 and 7 describe how the system was implemented and subsequently tested to ensure reliability, while Chapter 8 discusses its current limitations and the directions in which it could be extended. Chapter 9 concludes the report with a summary of project outcomes and a list of references.

The project was hosted within BISAG-N's research and development environment, which supports student teams from partner universities in gaining production-level experience across emerging technology domains. These include areas like artificial intelligence, geospatial analytics, and—as in this case—large-scale automated data collection and graph-based intelligence systems. The remainder of this chapter sets out the specific

details, purpose, scope, objectives, technology choices, and prior-art context of this developed project before the report moves on to project management and requirement analysis in Chapters 2 and 3.

1.1 Project Details

TABLE 1: PROJECT FACT SHEET

Fact / Detail	Description
Project Title	OSINT Collection from Social Media Network
Start Date	18th May 2026
End Date	13th July 2026
Internship Duration	8 Weeks (7 weeks active development, 1 week documentation & report submission)
Working Platform Name	Linkmap
Data Collection Target	X (formerly Twitter) public profile and timeline timelines
Relational Database	PostgreSQL 14+
Graph Database	Neo4j Desktop (Community/Enterprise)
Backend Framework	FastAPI (Python) with WebSockets
Frontend Technologies	HTML5, CSS3, Vanilla JavaScript (ES Modules), D3.js v7

This project is an end-to-end Open-Source Intelligence (OSINT) collection and analysis system designed to automate the discovery, collection, structuring, and visual exploration of publicly available social network activity on X (formerly Twitter). The platform targets a curated set of India-based political and media handles, continuously extracting tweets, profile metadata, hashtags, mentions, and shared links. It organizes the resulting dataset into two complementary databases—a PostgreSQL relational database for structured querying and a Neo4j graph database for relationship-centric analysis. A FastAPI backend exposes this data through a secured web interface and live data stream layer, which in turn powers a multi-page analyst dashboard offering handle intelligence, hashtag and topic analysis, coordinated-campaign detection, an interactive force-directed network graph, and an automated PDF intelligence-report generator.

Over the course of the internship, the pilot deployment of this system was pointed at twenty India-based political and media handles selected to span political parties, television news, digital journalism, and fact-checking organizations. A single representative scraping run collected 1,601 tweets across these twenty handles, from which the entity-extraction pipeline derived 303 distinct hashtags, 425 shared links, and twelve topic categories. The resulting network relationship graph for that run contained over three thousand two hundred nodes and more than six thousand relationships. These figures are

used throughout this report purely as an illustration of the pipeline's throughput on a realistic pilot dataset, and they are expected to grow substantially once the platform is pointed at a larger and more frequently refreshed list of handles.

The remainder of this section records the essential project metadata—including the title, host organization, team composition, guidance structure, and duration—summarized in Table 1 above. Meanwhile, Sections 1.2 through 1.6 expand on the motivation, scope, objectives, technology choices, and prior research context underpinning the system described in the rest of this report.

1.2 Purpose

The purpose of this project is to replace slow, manual, single-analyst OSINT workflows—which typically rely on browser-based searching, spreadsheet note-taking, and manual tracking of names and hashtags—with a repeatable, automated pipeline. This pipeline can be pointed at a set of public handles and produce a structured, query-able, and visually explorable intelligence dataset within hours rather than days. Manual investigation does not fail because analysts lack skill; it fails because human working memory and attention cannot reliably track hundreds of interlinked hashtags, mentions, and posting timestamps across dozens of accounts simultaneously. Automating the collection and structuring of this data, while leaving the higher-level interpretation to the human analyst, was therefore identified as the central purpose the system needed to serve.

A second, equally important purpose is pedagogical: the project was used as a vehicle for the student team to gain hands-on, production-grade experience across the full software stack required for a modern data platform. This includes browser automation, relational and graph database design, backend server engineering, real-time communication, frontend data visualization, and automated document generation—while working on a problem of genuine analytical value to BISAG-N. Rather than building isolated, disconnected exercises for each of these skills, the team deliberately chose a single problem large enough to require, and therefore genuinely exercise, every one of these competencies in an integrated way.

A third purpose was institutional: to leave BISAG-N with a reusable, extensible reference implementation of an OSINT pipeline that future student cohorts or research groups could extend. This could be done by pointing it at a different set of handles, adding new entity-extraction rules, or building additional analysis dashboards on top of the existing database schemas, rather than leaving a one-off, throwaway prototype. This informed a number of design choices described in later chapters, including the emphasis on a modular, clean backend route structure and a documented, versioned query library.

Finally, by combining a relational store with a native graph database, the project also serves as a practical demonstration of when and why graph databases outperform purely relational designs for social-network-shaped problems. Queries such as "which two handles most frequently share the same hashtag" or "which account bridges two otherwise disconnected clusters" are naturally expressed as graph relationships and would be

considerably more expensive to compute using relational table joins alone. Demonstrating this contrast concretely on a real dataset was itself part of the system's purpose.

1.3 Scope

The scope of the delivered system covers the complete lifecycle of an OSINT investigation for a defined set of public X (Twitter) handles. This includes targeted scraping, entity extraction, dual-database persistence, relationship-graph construction, campaign and anomaly detection, interactive visual exploration, and automated report generation. The following items were explicitly within and outside the scope of the eight-sprint development period.

In Scope

1. Automated collection of public profile and timeline data from a configurable list of X/Twitter handles using a stealth, anti-ban browser automation engine.
2. Extraction of hashtags, user mentions, shared links, and keyword-derived topics from public post text.
3. Persistent storage of raw and parsed data in a relational database (PostgreSQL), with conflict-safe deduplication.
4. Construction and incremental update of a property graph (Neo4j) representing users, hashtags, links, and topics along with their relationships.
5. A secured backend server exposing REST and WebSocket endpoints for all dashboard features.
6. A multi-page analyst web dashboard (including Overview, Handles, Network, Hashtags, Topics, Campaigns, Tweet Analysis, Scraper Control, Data Explorer, Intelligence, and the Nexus graph explorer).
7. A browser-accessible scraping control panel with live WebSocket progress feedback.
8. An automated, downloadable PDF intelligence report summarizing profile details, cross-handle relationship signals, and plain-English findings.

Out of Scope

1. Scraping or analysis of social networks other than X/Twitter (such as Facebook, Instagram, Telegram, or YouTube).
2. Deep natural-language analysis or machine-learning-based classification—topic tagging in the current system is based on keyword maps rather than complex artificial intelligence models.
3. Continuous, always-on streaming ingestion; data collection is executed as discrete, analyst-triggered jobs rather than a persistent live feed.
4. A dedicated mobile application—the dashboard is designed as a responsive web application accessed through standard desktop browsers.
5. Multi-user authentication and role-based access controls for the dashboard, which is currently intended for single-analyst use on a trusted workstation.

6. Formal deployment to a shared production cloud server; the system was developed and validated on local development workstations running PostgreSQL and Neo4j Desktop.

These boundaries were set deliberately at the start of the internship so that the eight available sprints could be spent building a complete, working pipeline end-to-end, rather than a partially-built system that attempted a wider scope. Items placed out of scope in this section are revisited as candidate future work in Chapter 7.

1.4 Objectives

This project was guided by the following primary objectives, each of which is traceable to a specific system module or feature described later in this report:

1. To design and implement an anti-ban, human-like data collection engine for X/Twitter, capable of reliably gathering profile and timeline data for a list of target handles without triggering rate limits or bot detection.
2. To build a robust, conflict-safe data persistence layer using a relational database, supporting deduplication and efficient batch writes.
3. To design and populate a property graph capturing the relationship structure between users, hashtags, links, and topics, suitable for network and campaign analysis.
4. To engineer an entity-extraction and topic-classification pipeline capable of parsing hashtags, mentions, links, and keyword-based topics from public posts.
5. To develop a secure, well-structured backend exposing all data through standard web interfaces and data streaming endpoints, hardened against common web-application vulnerabilities.
6. To build an analyst-facing web dashboard that visualizes handle intelligence, hashtag and topic trends, coordinated-campaign signals, and the full relationship graph in real time.
7. To implement an automated report generator that synthesizes database statistics and graph-topology signals into a single downloadable PDF document.
8. To validate the system against a real pilot dataset of India-based political and media handles and demonstrate its ability to surface genuine, human-interpretable findings.

Objectives 1 through 4 are addressed primarily through the scraper, storage and graph modules described in Chapters 4 and 5; Objectives 5 and 6 are addressed through the backend and frontend architecture described in Chapter 5 and implemented as described in Chapter 6.

1.5 Technology Stack Used

The platform, named Linkmap, was built entirely using open-source and freely available technologies, which kept this project free of licensing constraints and made it straightforward for all three team members to run the full stack independently on their own development workstations. Table 2 summarizes the primary technology choices

made at each layer of the stack; the paragraphs that follow explain the reasoning behind a number of the more consequential choices.

TABLE 2: TECHNOLOGY STACK SUMMARY

Layer / Component	Technology / Library	Description
Programming Language	Python 3.11	Core logic, backend, scraper, ETL pipeline
Browser Automation	Playwright (Chromium)	Stealth-patched automated data collection
Data Modelling & Validation	Pydantic v2	Input checks and data schemas
Relational Database	PostgreSQL 14+	Core system of record, structured data storage
Graph Database	Neo4j Desktop	Network relationship mapping and queries
Backend Framework	FastAPI (Starlette WebSockets)	REST API and real-time streaming endpoint
Frontend Layer	HTML5, Vanilla CSS, JavaScript (ES6)	Client-side dashboard (Single Page Application)
Data Visualisation	D3.js v7	Interactive force-directed network charts
Report Generation	ReportLab	Automated PDF intelligence report exports
System Utilities	loguru, ujson, aiofiles, tqdm	Color logs, faster JSON processing, async files
Version Control	Git	Project code history

Playwright was selected over other options like Selenium because it easily handles persistent browser contexts and automatically waits for pages to load. This is especially important for modern, JavaScript-heavy sites like X. Additionally, it provides native features to randomize browser characteristics, which helps the anti-bot-detection strategy described in Chapter 5.

Neo4j was selected instead of storing all network relationships in PostgreSQL because many of the platform's key queries—such as detecting shared hashtag campaigns, tracking mention chains, and identifying influential accounts—are naturally designed as relationship traversals. These are highly efficient to execute in a graph database but would require complex, slow table joins in a standard relational database.

FastAPI was chosen as the backend framework because it handles real-time, asynchronous connections efficiently, which is necessary for the live scraping progress updates shown on the dashboard. It also automates request data checks and generates self-documenting API endpoints, which simplified development as the dashboard expanded to include multiple pages.

The frontend dashboard was built using vanilla JavaScript without a complex framework. Given the project's eight-week timeframe, this approach minimized setup overhead and avoided build-tool complications. D3.js was used specifically for its flexibility in building the interactive force-directed network graphs that allow analysts to visually explore connections on the dashboard.

1.6 Literature Review

Open-Source Intelligence (OSINT) is defined as gathering and analyzing information from publicly available sources, such as news media, public records, and open social network sites. The standard OSINT lifecycle (planning, collection, processing, analysis, and dissemination) treats social network platforms as a rich but highly active source of data: the content is high-volume, changes quickly, and becomes valuable only when we can correlate it across multiple accounts and time periods. Existing research and methodologies in this space historically focus on manual, browser-based searches and advanced search engine queries. While these manual methods work well for looking up a single target, they do not scale when trying to monitor dozens of accounts over time.

Graph databases are a natural fit for social network analysis because they treat relationships as primary elements rather than secondary references. The property-graph model used by Neo4j represents entities (like users, hashtags, and links) as nodes, and the connections between them as edges. This makes multi-hop questions—such as finding accounts that bridge different discussion groups or identifying sudden co-occurrences of hashtags—easy to write as simple queries. Consequently, graph databases are widely used in social network analysis, mapping influence patterns, and detecting coordinated online behaviors.

Several commercial and open-source tools handle parts of the OSINT process separately. For example, some tools provide network visualizations but require data to be imported manually. Other software is excellent for analyzing static charts but cannot collect live data, and social network management tools offer real-time monitoring but lack a persistent relationship database or automated reports. The platform developed in this project, named Linkmap, stands out by integrating automated data collection, dual-database storage, relationship-graph generation, visual exploration, and report generation into a single pipeline. This pipeline is tailored specifically to monitor public communication and influence patterns among targeted accounts on X.

Another area of research focuses on detecting coordinated behavior on social networks. Studies in this field typically look for patterns like near-simultaneous posts sharing the same hashtags, tightly connected groups that mention each other frequently, or accounts

with abnormally high repost rates relative to their follower counts. The campaign detection features described in Chapters 5 and 7 of this report are designed in this spirit, serving as lightweight, database-optimized methods to spot these patterns without requiring heavy mathematical modeling.

The design of the platform was therefore informed by three areas of prior work: (i) standard OSINT methodologies, which shaped the platform's collection-processing-analysis-reporting workflow; (ii) graph database concepts for social network analysis, which shaped how database relationships and queries were modeled; and (iii) standard web application practices—such as database connection pooling, secure query structures, and real-time data streaming—which shaped the backend and frontend systems described in the chapters that follow.

2 Project Management

Delivering a multi-layered data platform—spanning browser automation, two databases, a backend API, a twelve-page frontend, and an automated reporting engine—within a fixed eight-week internship window required deliberate project management from the outset. With only three team members and no scope for schedule slippage, the team could not afford to discover major architectural problems late in the project; every week had to end with a working, demonstrable increment.

This chapter documents how that constraint was managed. Section 2.1 assesses the feasibility of this project along technical, schedule, operational, and implementation dimensions before development began. Section 2.2 describes how the work was planned once feasibility was confirmed—the chosen development approach, the resulting weekly milestones, how responsibilities were divided across the three-person team, and the dependencies that existed between those responsibilities. Section 2.3 then presents the overall project schedule.

Taken together, these sections explain not only what was built (which is the subject of Chapters 4 through 7) but how the team organized itself to build it reliably within a hard deadline, using an iterative process that allowed course-correction at weekly boundaries rather than only at the very end of the internship.

2.1 Feasibility Study

Before committing to the eight-week development plan described later in this chapter, the team informally assessed the feasibility of the proposed Linkmap platform along four standard dimensions: technical feasibility (could the team realistically build it with the skills and tools available), schedule feasibility (could it be built within the fixed internship window), operational feasibility (would the resulting system actually be usable by its intended analyst user), and implementation feasibility (was it achievable given the team size, hardware and institutional support available). This assessment was conducted informally, through discussion with the project guides rather than a separate formal feasibility document, but its conclusions directly shaped the scope decisions recorded in Section 1.3.

The overall conclusion of this assessment was that the project was feasible provided it was built incrementally, with each week's scope kept small enough to be fully completed and integration-tested before moving to the next—an approach that is reflected in the sprint-based methodology described in Section 2.2.1 and in the weekly milestones outlined in Section 2.2.2. The four individual dimensions of the feasibility assessment are discussed in turn in Sections 2.1.1 through 2.1.4.

2.1.1 Technical Feasibility

Technical feasibility was assessed by matching the skills required against the skills already available within the team, and the maturity of the candidate tools against the requirements identified in Chapter 3. All three team members entered the internship with

a working background in Python, which underpins every layer of the backend, scraper, and graph-transformation code; this significantly reduced the ramp-up time that would otherwise have been needed before productive development could begin. None of the three members had prior production experience with Playwright, Neo4j, or FastAPI specifically, so a portion of Week 1 was deliberately allocated to structured learning of Neo4j, Cypher, and OSINT methodology before implementation began, rather than attempting to learn these tools while simultaneously building against a deadline.

The principal technical risk identified at the outset was the inherent fragility of selector-based data collection: X's front-end markup is not a published, versioned API, and any structural change to its elements can silently break selectors. This risk was judged acceptable rather than disqualifying, because it could be mitigated through defensive coding practices—multi-strategy selector fallbacks, resilient parsing, and clear logging of extraction failures—all of which were in fact implemented during Week 1 and refined further during the Week 3 security and bug-fixing sprint described in Chapter 7.

A second technical consideration was whether a three-person team could realistically maintain two databases, a backend, and a frontend dashboard simultaneously without one layer's changes silently breaking another. This was judged feasible on the basis that the team's three functional roles (Section 2.2.3) cleanly mapped onto the system's natural layers, keeping most day-to-day changes localized to a single owner even when the layers themselves were tightly coupled.

On balance, all of the individual technologies used were mature, well-documented, and either free or already licensed for use at BISAG-N, and the team's combined skill set was judged sufficient to acquire the remaining, unfamiliar pieces (Playwright, Neo4j, FastAPI, D3.js) within the time available. The project was therefore judged technically feasible.

2.1.2 Time Schedule Feasibility

Time schedule feasibility was assessed against a fixed constraint: the internship duration was set externally at eight weeks (18th May to 13th July 2026), of which seven weeks were allocated to active development and the remainder to consolidation, testing, and this report. Given that constraint could not be renegotiated, the feasibility question was not "is eight weeks enough time in the abstract" but "is the proposed scope achievable in eight weeks if broken into weekly increments."

The team addressed this by deliberately sequencing the scope so that each week's deliverable was independently valuable and independently demonstrable, rather than planning for a single "big bang" integration at the end of the internship. Week 1 targeted a working, if minimal, end-to-end pipeline (scrape, store, graph, and query) precisely so that schedule risk in later weeks would be limited to extending an already-working system rather than assembling one from scratch under time pressure.

In practice, this schedule assumption was validated retrospectively: each of the eight planned weekly milestones was delivered on schedule, including the Week 3 sprint that was deliberately reserved for a full security and defect audit rather than new features—a

schedule decision made possible by the fact that the preceding two weeks had left enough slack to absorb a maintenance-focused week without delaying the Week 4 network explorer milestone. The schedule was therefore judged, and subsequently confirmed, to be feasible.

2.1.3 Operational Feasibility

Operational feasibility concerns whether the finished system can actually be operated, day to day, by its intended user—an OSINT analyst who is not necessarily a software engineer—rather than only by the developers who built it. This was assessed against the user characteristics described in Section 3.3: the target analyst needs to be able to start a new scrape job, read the resulting dashboard, and generate a report without needing to write Cypher, SQL, or Python.

The team judged this feasible provided the dashboard hid all direct database interaction behind a browser user interface, which shaped several concrete design decisions described in Chapter 5: the Scraper Control Panel exposes a simple text input and a tweet-limit slider rather than a command-line interface; filter bars across the data-heavy pages use dropdowns and text inputs rather than requiring raw query syntax; and the PDF report generator condenses Cypher- and SQL-derived findings into a single downloadable document with plain-English inference sentences, so that a non-technical stakeholder reading the exported report does not need direct dashboard access at all.

A secondary operational consideration was the analyst's tolerance for waiting: because a full data collection run can take on the order of ninety minutes for twenty handles, the live progress feed and per-handle preview cards were treated as an operational necessity rather than a cosmetic feature, so that an analyst could distinguish a slow-but-healthy scrape from a stalled or failed one without needing to inspect server logs. On this basis, the project was judged operationally feasible for its intended single-analyst usage pattern.

2.1.4 Implementation Feasibility

Implementation feasibility was assessed against the practical resources available to the team: three developers, standard development workstations, and BISAG-N's institutional infrastructure and mentorship. Because the entire technology stack used is free and open-source, the project carried no software licensing cost, which removed a common source of implementation risk for student projects.

Hardware requirements (detailed in Section 3.6) were modest enough to be met by standard laptop-class workstations available to each team member, allowing all three members to run the full stack—Chromium, PostgreSQL, Neo4j Desktop, and the backend server—locally and in parallel, rather than depending on shared or centrally-provisioned infrastructure that could have introduced scheduling bottlenecks.

Institutional support from BISAG-N, in the form of regular guidance from the external project guide and access to the institute's established project review processes, provided a further feasibility safeguard: the weekly progress-report cadence described in Section

2.2.1 gave the team a natural checkpoint at which technical or schedule risks could be raised with the guides before they escalated into blocking issues.

Taken together—zero licensing cost, modest hardware needs, a three-person team whose skills matched the required roles, and consistent institutional mentorship—the project was judged implementation-feasible at the outset, a judgement subsequently borne out by the on-schedule delivery record.

2.2 Project Planning

With feasibility confirmed along all four dimensions in Section 2.1, the team moved to concrete planning. Four planning questions needed to be answered before development could begin: what development process would be followed (Section 2.2.1); what the specific weekly deliverables would be (Section 2.2.2); who on the team would own which parts of the system (Section 2.2.3); and how those ownership areas depended on one another (Section 2.2.4).

The guiding planning principle, consistent with the feasibility conclusions in Section 2.1.2, was that every week needed to end in a working, demonstrable increment rather than a partially-finished feature. This principle is reflected throughout the plans described in this section: the weekly milestones are each independently functional systems (even if incomplete relative to the final platform), and the role assignments were chosen specifically so that no single week's plan depended on more than one person's unfinished work at a time wherever this could be avoided.

2.2.1 Development Approach & Justification

The project followed an iterative, sprint-based development approach, with each of the eight weeks treated as a self-contained sprint delivering a working increment of the platform. This approach was chosen over a traditional waterfall model for three reasons. First, the project spanned a wide range of genuinely dependent layers—scraper, storage, graph, backend, frontend, and reporting—and building each layer only far enough to unblock the next allowed early integration risk (for example, schema mismatches between the scraper's output and the Neo4j loader) to surface within days rather than at the end of the project. Second, an incremental approach allowed the team to validate each new page or feature of the dashboard against real, live-scraped data as soon as it was produced, rather than working against synthetic or assumed data shapes. Third, weekly sprint boundaries provided a natural cadence for internal code review, integration testing, and bug triage, which proved valuable during the dedicated security-and-bug-audit sprint in Week 3.

Within each sprint, work was further divided along the three team roles described in Section 2.2.3, with daily informal synchronization to keep the scraper's output schema, the database schemas, and the frontend's expected API contract mutually consistent as the system grew. Each sprint concluded with a short internal review against the weekly milestones, and a round of integration testing across the full pipeline before the next sprint's work began.

This approach was justified retrospectively by two observations. First, several significant defects—including the frontend file-download interception bug and the Neo4j database schema updates—were caught and fixed within the same or following sprint precisely because the team was continuously integrating against live data rather than deferring integration to the end of the project. Second, the dedicated Week 3 audit sprint, which resolved sixteen defects across the codebase (see Chapter 7), would not have been schedulable under a waterfall plan without displacing feature work; under the sprint-based plan, it was simply treated as one sprint's priority among several.

2.2.2 Milestones & Deliverables

The major milestones delivered at the end of each development sprint are outlined below. Each milestone was demonstrated and reviewed internally before the following sprint's work began.

TABLE 3: WEEKLY MILESTONES & DELIVERABLES

Sprint	Duration	Key Milestone Delivered
Week 1	18–25 May 2026	Foundational data collection pipeline (Playwright and anti-ban rate limiting), initial Neo4j graph schema, and a first full 20-handle pilot scrape (1,601 tweets) with baseline analysis queries.
Week 2	25–31 May 2026	PostgreSQL database integration with cross-run deduplication, migration to a tweet-free Neo4j schema, a full backend server configuration, and the first 7-page analyst dashboard.
Week 3	1–7 June 2026	Real-time WebSocket scraper control panel, Data Explorer and Intelligence analysis pages, and a comprehensive security and code audit resolving 16 bugs across the codebase.
Week 4	8–14 June 2026	The network interactive graph explorer—a dedicated D3.js force-directed graph merging PostgreSQL and Neo4j data with on-demand node expansion.
Week 5	15–21 June 2026	Date-range handle selector and the first automated PDF report generator, plus a critical fix to frontend file-download interception.
Week 6	22–28 June 2026	Autocomplete handle search, and resolution of six structural bugs (dashboard layout, simulation timing, DOM scoping, and positioning) that stabilized the graph explorer page at scale.
Week 7	29 June–5 July 2026	Enhanced PDF report containing full tweet-by-tweet logs, four database relationship tables, and an expanded, graph-aware inference engine.

Sprint	Duration	Key Milestone Delivered
Week 8	6-13 July	Project documentation, formatting, final reviews, and final report submission.

A consistent pattern is visible across all eight milestones: each week combined a substantial new capability with incremental hardening of what had already been built. For example, the network explorer developed in Week 4 relied on the backend server and connection-pooling patterns established in Week 2 and hardened in Week 3. This layering is a direct consequence of the iterative development approach and is discussed further in the module descriptions of Chapter 4 and the implementation record of Chapter 6.

2.2.3 Roles & Responsibilities

The three-member team adopted fixed functional roles for the duration of the project. While each member owned a primary layer of the system, all three collaborated closely on integration points such as the data schema and the API contract between the backend and frontend.

TABLE 4: ROLES & RESPONSIBILITIES

Member	Role	Primary Responsibilities
Rishav Pal	Data Engineer	Browser automation, rate limiting, PostgreSQL schema, database connection pools, backend server routing, and system bugs audit.
Sayantan Mandal	OSINT Analyst	Target account selection, visual dashboard frontend layout, PDF report templates, and narrative analysis interpretation.
Shivam Kumar	Graph Analyst	Neo4j property graph schema, Cypher analytical queries, graph-transformation code, D3.js force layouts, and network metrics.

These roles were assigned at the start of the internship based on each member's prior coursework and stated interests, and they were kept stable throughout the eight sprints. Stability of ownership proved important for accountability during the Week 3 audit sprint in particular, where each of the sixteen identified defects could be routed directly to the team member who owned the affected file without ambiguity.

2.2.4 Group Dependencies

Because the platform is a tightly-coupled pipeline, the three roles carried explicit mutual dependencies. The Graph Analyst's Neo4j schema decisions (for example, the Week 2 migration to a tweet-free graph schema) directly required corresponding changes to the Data Engineer's graph-transformation code and to frontend graph-rendering logic. The OSINT Analyst's frontend pages depended on stable, documented API contracts from the

backend route files, which meant route signatures were treated as an informal contract that could not change without prior notice within the team. Finally, the PDF report feature (Weeks 5 and 7) depended on both the PostgreSQL and Neo4j databases being populated and reachable, which motivated the graceful-fallback design discussed in Chapter 4, ensuring that an issue in one database did not block the entire reporting feature.

TABLE 5: INTER-ROLE DEPENDENCY MATRIX

Source Role	Target Role	Nature of Dependency
Graph Analyst	Data Engineer	Neo4j schema modifications require matching updates to the graph transformer script (<code>transformer.py</code>).
Data Engineer	OSINT Analyst	Frontend dashboard views consume backend endpoints; route shapes must be communicated before updates.
Graph Analyst	OSINT Analyst	Report inference rules depend on specific relationship attributes and queries defined in the graph library.
Data Engineer	Graph Analyst	PostgreSQL column values and types determine the input format of the CSV extractor scripts.

2.3 Project Scheduling (Gantt/PERT Chart)

The eight-week schedule was tracked informally against calendar weeks rather than a formal PERT network, given the small team size and the sequential, pipeline-shaped nature of the dependencies between data collection, storage, graph, and presentation layers. Nonetheless, the schedule does exhibit a clear critical path: the data collection and storage work of Weeks 1–2 gates every subsequent week, since the backend (Weeks 2–3), the graph explorer (Week 4), and the reporting feature (Weeks 5 and 7) all consume data produced by that early pipeline. Within this constraint, the security and bug audit sprint in Week 3 was deliberately scheduled after the first working backend and dashboard existed, so that the audit would have real, integrated code to examine rather than isolated modules.

A small amount of schedule float existed around the Week 6 stabilization sprint, which addressed bugs introduced by the network explorer and reporting features; had these bugs been more severe, the float built into Week 6 would have absorbed the additional time without displacing the Week 7 milestone. In the event, this float was used largely as planned, and the project completed all eight milestones within the originally allocated eight-week window.

A Gantt chart representation of the schedule, showing the sprint durations, dependency paths, and the float around Week 6, is to be inserted in the final document.

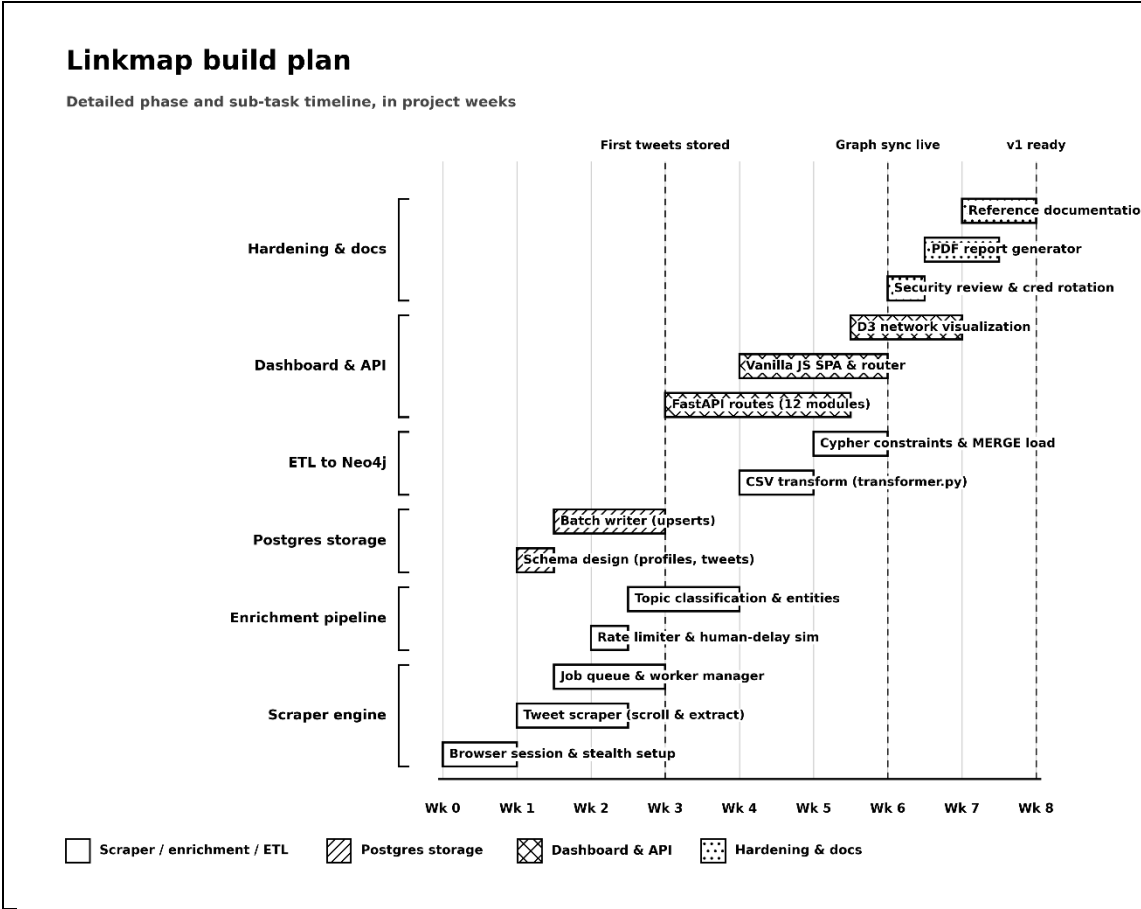


FIGURE 1: PROJECT GANTT CHART (18 MAY 2026 – 13 JULY 2026)

The chart in Figure 2.1 should be read alongside Table 2.1: each bar corresponds to one of the eight weekly sprints, and between bars correspond to the producer/consumer relationships listed in Table 2.3.

3. System Requirement Study

Before any code was written, the team carried out a requirement study to establish precisely what the proposed system needed to do, for whom, and under what constraints. This chapter documents that study. It begins by describing the existing approach to social network OSINT work and its limitations (Sections 3.1 and 3.2), before characterizing the system's intended users (Section 3.3). It then sets out the functional and non-functional requirements the system needed to satisfy (Sections 3.4 and 3.5), the hardware and software environment it needed to run within (Section 3.6), the constraints under which it had to operate (Section 3.7), and finally the assumptions and external dependencies the design relies upon (Section 3.8).

This requirement study was conducted iteratively alongside the early development sprints described in Chapter 2, rather than as a single up-front document frozen before implementation began—a natural consequence of the sprint-based methodology adopted for this project. Nonetheless, the requirements recorded in this chapter reflect the stable understanding of the problem that had emerged by the end of Week 1, and against which the remaining sprints of development were subsequently measured.

3.1 Existing System Overview

Prior to this project, OSINT investigation of Indian political and media narratives on X was carried out through a combination of generic, point-in-time tools. These included manual browsing and TweetDeck-style monitoring columns for real-time awareness; advanced search engine queries for background research on individual targets; and, where relationship mapping was required, general-purpose link analysis tools into which data had to be manually imported. Investigators would typically record findings in spreadsheets or note-taking documents, manually cross-referencing hashtags and mentions across handles by eye.

This existing approach is best characterized as a set of independent, manually operated tools rather than a unified system in the engineering sense. Each tool addresses one narrow task (searching, monitoring, or graphing) and none of them share a common, persistent database model. An investigator trying to determine, for example, whether two handles are coordinating their hashtag usage would need to manually collect posts from both accounts, manually identify shared hashtags, and manually judge whether the overlap was coincidental or deliberate—a process that does not produce any reusable data once the investigation concludes.

3.2 Limitations of the Existing System

The manual, tool-fragmented approach described in Section 3.1 carries several concrete limitations that directly motivated this project:

1. Entirely manual data collection: There is no scheduled or repeatable data gathering, making it impractical to monitor more than a handful of handles over time.

2. No persistent relationship database: Each investigation effectively started from a blank slate because there was no historical relationship graph.
3. No automated campaign detection: Coordinated behaviors (such as hashtag usage bursts or shared posting patterns) had to be spotted manually, which was easy to miss at scale.
4. Fragmented tools: Analysts had to use separate applications for monitoring, searching, and charting, with no shared data model between them.
5. No standardized reporting: Findings existed only as informal, manual notes rather than structured reports.
6. Poor scalability: The manual approach did not scale beyond a small number of accounts or a brief time window.
7. No data audit trail: There was no audit trail linking a finding back to the specific post, timestamp, and handle that produced it, making it difficult to verify or revisit a conclusion later.

Each of these limitations maps directly onto a corresponding functional requirement in Section 3.4: for example, the lack of a persistent relationship graph motivates the Neo4j-backed requirements (FR3–FR4), and the lack of a shareable reporting artefact motivates the PDF report-generation requirement (FR8).

3.3 User Characteristics

This system is designed around two user profiles: the primary user (the OSINT Analyst) and the secondary user (the Data/Graph Engineer).

Primary User Profile (OSINT Analyst / Intelligence Investigator)

1. **Role and Expertise:** Focuses on analyzing public communication, political narratives, and media patterns. This user is domain-knowledgeable about political and media networks in India but does not necessarily have a deep technical background in coding.
2. **Technical Proficiency:** Low to medium. Comfortable navigating web browsers and visual interfaces, but unfamiliar with raw query languages or system configuration.
3. **Primary Interface:** The multi-page web dashboard, utilizing clear visual panels, search bars, and filter dropdowns rather than command lines.
4. **Frequency of Use:** Periodic. Triggers data collection runs and reviews relationship findings as investigations require.
5. **Key Needs:** Fast, highly visual, and easily exportable intelligence summaries (such as downloadable PDF reports).

Secondary User Profile (Data/Graph Engineer)

1. **Role and Expertise:** Responsible for maintaining and extending the data ingestion pipeline, modifying target handle configurations, monitoring system health, and making updates to the database schemas.
2. **Technical Proficiency:** High. Fully comfortable working with Python scripts, SQL tables, Cypher graph queries, and backend configuration settings.
3. **Primary Interface:** Direct codebase configuration files, backend route scripts, and raw database query panels.
4. **Frequency of Use:** Occasional. Focuses on system maintenance, schema updates, and onboarding new handles or features.
5. **Key Needs:** A stable, well-documented, and extensible code architecture.

TABLE 6: PRIMARY VS SECONDARY USER CHARACTERISTICS

Characteristic	Primary User (OSINT Analyst)	Secondary User (Data/Graph Engineer)
Technical proficiency	Low to medium—comfortable with web browsers, unfamiliar with raw database queries.	High—fully comfortable with Python, SQL, Cypher, and configuration scripts.
Primary interface	Twelve-page visual web dashboard.	Code files, backend routes, and database consoles.
Frequency of use	Periodic—runs collection jobs and analyzes narratives as needed.	Occasional—focuses on system updates and schema maintenance.
Key need	Clear, visual summaries and exportable PDF intelligence reports.	A stable, documented, and easily extensible architecture.

3.4 Functional Requirements

The following functional requirements were identified during the requirement study and used as the baseline against which the eight development sprints described in Chapter 2 were planned:

1. **Targeted Data Collection:** The system shall scrape profile metadata (display name, biography description, follower counts, following counts, and verified badges) and public post timelines for each configured account handle.
2. **Relational Ingestion:** The system shall persist all collected profiles and timeline posts in a structured relational database (PostgreSQL) using idempotent, conflict-safe upsert procedures to prevent duplicate records.

3. **Entity Extraction:** The system shall parse and extract hashtags, user mentions, external web links, and keyword-derived topic tags from the text of every gathered post.
4. **Graph Construction:** The system shall construct and update a property graph (Neo4j) representing users, hashtags, links, and topics, mapping the direct relationships between them.
5. **API Layer and Streaming:** The system shall expose all collected datasets through documented REST endpoints and shall stream live progress logs for active data collection runs via WebSockets.
6. **Dashboard Visualization:** The system shall provide visual dashboard screens for account profiles, hashtag analysis, topic groupings, campaign detection, post search, interactive network graphs, and free-form database queries.
7. **Interactive Scraper Controls:** The system shall allow an analyst to input target handles, set collection limits, start a new run directly from the browser, and monitor real-time progress.
8. **Automated Intelligence Reports:** The system shall allow an analyst to select multiple handles and dates to generate a downloadable PDF report summarizing profile statistics and cross-handle relationship patterns.
9. **Coordinated Activity Detection:** The system shall automatically identify coordinated campaigns, including temporary hashtag co-occurrence spikes and accounts that frequently distribute identical links or hashtags.
10. **Graceful System Degradation:** The system shall degrade gracefully if the graph database is temporarily offline, continuing to serve all relational database features without crashing.

These ten requirements were traced individually against the delivered features, and each is discussed again, from an implementation perspective, in Chapters 5 and 6.

3.5 Non-Functional Requirements

Beyond the functional requirements listed in Section 3.4, the following non-functional qualities were treated as first-class requirements throughout development, reflecting lessons learned during the Week 3 security and performance audit described in Chapter 7:

1. **Performance:** Bulk database writes are batched (reducing a full 20-handle scrape from roughly 2,000 sequential database connections to 20 batched writes). All relational database access is routed through a shared connection pool instead of opening new database connections on every request.
2. **Security:** All database queries are fully parameterized to protect against injection vulnerabilities. Dynamic sorting parameters are validated against strict whitelists of allowed columns, and all incoming requests are verified via validation schemas.

An in-memory, IP-based rate limiter protects the API backend, while security headers are enforced at the middleware layer.

3. **Reliability:** Every graph database query used in combined endpoints is wrapped to handle errors gracefully. If the graph database goes offline, the platform degrades gracefully to return empty graph lists instead of failing the entire request. Cross-run deduplication prevents duplicate posts from being stored during repeated runs.
4. **Usability:** Dashboard pages use skeleton animations while data is loading to avoid blank screens. Standard filter configurations are placed across all search views, and an autocomplete search index is implemented to let analysts quickly look up handles in large graph structures.
5. **Maintainability:** The backend codebase is organized cleanly into separate files per feature area (such as dashboard metrics, hashtag analysis, and coordinated campaign detection). All database interactions follow a consistent connection pool pattern.
6. **Availability:** The scraping engine, backend server, and frontend client can be launched independently using a single orchestrator command. The dashboard displays the health status of both database systems via a dedicated status check.
7. **Portability:** The technology stack depends only on freely available, cross-platform components (Python, PostgreSQL, Neo4j, and modern web browsers) and was successfully developed and verified on both Windows and Linux workstations.

3.6 Hardware & Software Requirements

The following details outline the minimum hardware and software environments required to run the complete stack—including the browser scraping tools, the PostgreSQL database, the Neo4j database, and the backend server—on a single development workstation, which is how this system was built and validated throughout this project.

TABLE 7: HARDWARE & SOFTWARE REQUIREMENTS

Category	Minimum Specification	Recommended Specification
Processor	Intel Core i5 (8th Generation) or equivalent	Intel Core i7 / AMD Ryzen 7, or higher
RAM	8 GB	16 GB (for concurrent databases & browser runs)
Storage	10 GB free space (HDD)	25 GB free space (SSD recommended)
Operating System	Windows 10 or Ubuntu Linux 20.04	Windows 11 or Ubuntu Linux 22.04+
Runtime	Python 3.11	Python 3.11
Databases	PostgreSQL 14, Neo4j Desktop	PostgreSQL 15+, Neo4j Desktop
Browser (Dashboard)	Chrome, Edge, or Firefox	Chrome or Firefox (latest version)
Version Control	Git	Git

These requirements reflect a single-workstation deployment model. Chapter 7 discusses the additional infrastructure (such as dedicated database servers and load balancing) that a shared, multi-analyst environment would require beyond this baseline.

3.7 Constraints

The requirements described in Sections 3.4 through 3.6 had to be met within a number of constraints imposed either by the target social network (X/Twitter), the chosen technology stack, or the nature of the OSINT domain itself. The following subsections record these constraints under five headings: user interface, communication interface, hardware interface, application criticality, and safety/security.

These constraints were treated as design inputs from the very first sprint rather than as an afterthought. In particular, the safety and security constraints described in Section 3.7.5

directly motivated the dedicated audit sprint in Week 3 (described in Chapter 2), which resolved a number of defects that, left unaddressed, would have violated these constraints.

3.7.1 UI Constraints

The dashboard is a browser-based single-page application built with vanilla JavaScript (ES Modules) and D3.js, with no frontend framework dependency. It targets modern evergreen browsers only and does not support legacy browsers like Internet Explorer. All pages share a single design system (including typography, color tokens, and spacing parameters) enforced through shared CSS stylesheets to keep the visual presentation consistent. A further constraint, discovered during Week 6, is that any client-side single-page application router must explicitly exclude file downloads, binary blobs, and download-attribute links from its navigation-interception logic, or file-download features (such as CSV data or PDF report exports) will silently fail; this constraint is now documented for any future views added to the system.

3.7.2 Communication Interface

All client-server communication occurs over HTTP(S) for standard request/response API calls, and over WebSocket for two live channels: data collection progress broadcasting and database-ready notifications after a run completes. All payloads are JSON-formatted. WebSocket connections must be closed cleanly on job completion to avoid memory leaks on the server, and the server must automatically detect and drop references to disconnected clients rather than continuing to attempt delivery to them.

3.7.3 Hardware Interface

The system has no direct interface to specialized physical hardware like sensors, industrial controllers, or dedicated networking appliances; all hardware interactions in this project are limited to the standard computing and network resources of the workstation on which it runs. This nonetheless carries a genuine design constraint: because the browser scraper, the relational database server, the graph database server, and the backend server are all expected to run concurrently on the same workstation in the current single-machine model, the combined memory and CPU footprint of these processes is a real constraint on the hardware specifications listed in Section 3.6.

Specifically, browser memory usage scales with the complexity of the pages being loaded. Because the scraper maintains a persistent browser context open for the duration of a collection job, memory pressure during a long-running scrape was identified early as a constraint. This is one of the main reasons the rate limiting system (described in Chapter 5) enforces cool-down intervals between target accounts, rather than collecting data from multiple handles in parallel.

A second constraint concerns network traffic: because data collection is outbound web traffic to the social network over the workstation's network interface, no special hardware is required beyond a stable internet connection. However, the rate-limiting and scroll pacing described in Chapter 5 was designed to keep outbound request volume within a range that mimics an ordinary human browsing the same accounts.

Should the platform be moved from the current single-workstation deployment model to a multi-server setup in the future—discussed as a future direction in Chapter 7—the hardware-interface constraints would shift from sharing one machine's resources to network latency and connection-pool configurations between separately hosted services, requiring these settings to be adjusted for a distributed layout.

3.7.4 Criticality of Application

The application is not life- or safety-critical, but it is analytically and reputationally sensitive: it collects and republishes information about public accounts, and any errors in data extraction, deduplication, or relationship mapping could affect an analyst's findings. The correctness of data lineage (which post, from which account, at which time, produced a specific entity) is therefore treated as a high-priority quality attribute throughout the design. This is the reason every record in the relational database retains its source handle and timestamp, and every graph database relationship preserves the metadata needed to trace it back to the underlying post-level evidence.

3.7.5 Safety & Security Considerations

Because the platform collects and stores data about public accounts and is accessed through a web interface, the following safety and security considerations were treated as design constraints:

1. **Parameterized Queries:** All database queries are fully parameterized; no user-controllable input is ever written directly into a query string.
2. **Input Validation:** Sorting and query parameters exposed to the API are checked against explicit whitelists rather than accepting free-form text.
3. **Rate Limiting:** An in-memory rate limiter is applied to all backend routes to prevent system abuse.
4. **Responsible Collection:** The scraper targets only publicly visible profile and timeline data, applying human-like scroll pacing and intervals between accounts to avoid putting unnecessary load on the target social network.
5. **Secure Credentials:** Login credentials for the scraping account are read from environment configurations rather than being hard-coded in the source files, and session cookies are stored locally to avoid re-authenticating on every run.
6. **Security Middleware:** Access rules are explicitly configured, and security headers are applied at the server middleware layer to every response.

3.8 Assumptions & Dependencies

The requirements and constraints described above rely on the following assumptions and external dependencies, which were treated as risks to be monitored rather than guarantees:

1. It is assumed that valid, authenticated cookies are available to the data collection engine at run time.
2. It is assumed that the target handles are public accounts whose profiles and timelines are visible without requiring special follow relationships or access approvals.

3. The system depends on the continued structural stability of the social network's web frontend; selector-based data collection is naturally sensitive to changes in the target site's document structure.
4. It is assumed that both databases are reachable from the backend server; the system is designed to degrade gracefully, but cannot function fully, if they are completely offline.
5. It is assumed that the analyst's workstation has sufficient resources to run the browser scraper, relational database, graph database, and backend server concurrently, as discussed in Section 3.7.3.
6. This project depends on the continued availability and current licensing of the open-source software libraries listed in Section 1.5.

4 Proposed System Requirements

Having established what the system needed to do (Chapter 3), this chapter describes the system that was actually proposed and subsequently built to satisfy those requirements. The platform, named Linkmap, was designed as a six-stage pipeline, with each stage corresponding to one of the modules described in Section 4.2:

1. Targeted scraping of profile and timeline data from the social network (X/formerly Twitter) using browser automation tools.
2. Entity extraction and topic classification of the raw post text.
3. Persistence of structured data into PostgreSQL with cross-run deduplication.
4. Transformation of the accumulated dataset into a Neo4j property graph via a dedicated graph-transformer and Cypher loader.
5. Exposure of both stores through a secured backend server offering REST and WebSocket access.
6. Presentation through a twelve-page analyst dashboard, culminating in the Nexus interactive graph explorer and an automated PDF report generator.

Each of these six stages was designed to be independently testable and, where possible, independently useful—a design choice that mirrors the iterative development approach described in Section 2.2.1. For instance, the scraping and storage stages alone (stages 1–3) already produce a usable, query-able database dataset even before any graph or dashboard work is layered on top of them, which is precisely the order in which the system was built across the first two development sprints described in Chapter 2.

The proposed system deliberately treats the two databases as complementary rather than redundant: PostgreSQL is the system of record for raw, high-volume posts and profile data and is optimized for filterable, paginated queries (as used throughout the search and explorer views described in Section 4.3), while Neo4j is a derived, periodically rebuilt view of the same underlying data optimized for relationship traversal (as used throughout the network explorer views, and the campaign-detection queries described in Chapter 5). This separation of concerns is a recurring theme throughout the remainder of this report.

Because the graph store is derived rather than authoritative, the proposed system was also required to remain fully functional—for all database-backed features—even when the graph database is temporarily unreachable. This graceful-degradation requirement, first identified during the Week 4 design of the graph explorer, is implemented consistently across every backend route that merges data from both stores, and is discussed further in Section 4.2's description of the Backend Module.

4.1 Overview of Proposed System

At a high level, the proposed system functions as an integrated pipeline. The analyst interacts with the dashboard using standard web requests and real-time data streaming. A

backend server coordinates all access to the databases (PostgreSQL and Neo4j). An asynchronous scraping pipeline (utilizing a job queue, background worker threads, automated browser sessions, parsing utilities, and database writers) runs independently. This pipeline collects data without locking the user interface and broadcasts real-time progress updates back to the dashboard. The design details in Chapter 5 expand this overview down to individual software functions and files.

At this overview level, three key properties of the proposed system are worth highlighting before the detailed module descriptions in Section 4.2:

1. **Asynchronous Pipeline:** First, the system is asynchronous by design. Data collection runs in a background worker thread, meaning the dashboard remains fully responsive and other information pages remain accessible while collection is in progress.
2. **Database Cache:** Second, the system is cache-aware. A lightweight in-memory cache stores repeated read queries to reduce database load.
3. **Live Observations:** Third, the system is observable. Every stage of the pipeline, from job creation to the final graph load, broadcasts status updates that the dashboard displays as a readable progress feed, which directly satisfies the operational requirements of the system.

4.2 Module Descriptions

1. Scraper Module

Consists of the browser automation tools that handle session authentication, rate-limiting rules to prevent blocks, account profile and timeline collection, and the text parsing pipeline that extracts hashtags, user mentions, external links, and keyword topics. This module manages all direct interactions with the target social network.

2. Storage Module

Consists of the relational database layer (PostgreSQL) which provides conflict-safe storage for account profiles and post records, a database connection client, and the deduplication rules that ensure repeating runs do not save duplicate posts. This module serves as the core system of record for all collected data.

3. Graph Module

Consists of the graph transformation utilities that convert the accumulated relational database records into CSV files representing network nodes and relationship edges. It also includes the loaders that import these CSV files into the graph database (Neo4j), alongside the queries used to detect campaign activities. This module runs after data collection to ensure the graph database is synchronized.

4. Backend Module

A backend application server that organizes specific endpoints for handles, hashtags, topics, campaigns, and network views. It shares a common rate limiter,

an in-memory cache, and connection pools to both databases, serving as the single access point for the dashboard interface.

5. Frontend Module

A multi-page web dashboard built with vanilla JavaScript (including views for metrics, networks, topics, campaigns, search, and scraper controls). This module serves as the primary user interface that analysts interact with directly.

6. Reporting Module

The automated report generator that merges database statistics and graph relationship networks to output formatted PDF intelligence reports, summarizing findings in plain English for analysts to export.

These six modules correspond directly to the six pipeline stages introduced at the start of this chapter, and their detailed internal design—including the specific classes, functions and queries each one contains—is described in full in Chapter 5.

4.3 System Features

Building on the six modules described in Section 4.2, the proposed system exposes the following analyst-facing features through its twelve-page dashboard:

TABLE 8 : SUMMARY OF DELIVERED SYSTEM FEATURES

Feature	Description
Scraper Control Panel	Dashboard view for setting collection limits, launching scraping runs, and monitoring progress via live WebSocket logs.
Handle Intelligence	A sortable list of tracked accounts displaying follower counts, classifications, post counts, and mention levels.
Influence Network	A D3.js force-directed visualization of database relationships with query filters and link/hashtag toggles.
Hashtag Analysis	Rankings of top hashtags, co-occurrence pairs, and coordinated hashtag alerts.
Topic Clusters	Groups target accounts and hashtags into topic clusters based on keyword taxonomy mapping.
Campaign Detection	Flags coordinated hashtag burst campaigns and visualizes them using radial relationship maps.
Tweet Analysis	Search page supporting keyword and account searches over all collected posts with CSV data export.
Data Explorer	In-depth search console with advanced filters for domain sharing, hashtags, and engagement metrics.
Intelligence	Traverses databases to calculate network metrics (such as bridge accounts, mention flows, and anomalous behaviors).
Nexus Graph Explorer	Split-screen visualizer combining autocomplete searches, interactive node expansions, and profile side panels.

Feature	Description
Automated PDF Reporting	Generates exportable PDF files summarizing account engagement charts, network relations, and plain-English insights.

Together, these eleven features fully satisfy the ten functional requirements listed in Section 3.4.

4.3 Advantages of Proposed System

The design of this system yields several major advantages over the manual approach previously used:

- 5 End-to-end automation removes the manual correlation burden from the analyst, allowing a full collection and analysis cycle for the target accounts to complete in under two hours.
- 6 The dual-database architecture combines the query flexibility of a relational database with the native relationship traversal strength of a graph database, avoiding the trade-offs of forcing both workloads into a single store.
- 7 Graceful degradation ensures every graph-dependent feature falls back to relational data rather than failing outright, keeping the dashboard fully usable even when the graph database is temporarily offline.
- 8 Live visibility into running data collection jobs is provided through real-time progress feeds, removing the uncertainty of background scraping scripts.
- 9 Reproducible, exportable intelligence in standard data formats (such as CSV and PDF files) allows findings to be easily shared with stakeholders who do not have direct access to the dashboard.
- 10 A security-hardened backend (featuring parameterized queries, whitelisted sort columns, rate limiting, and access control headers) resulting from a dedicated review sprint that resolved sixteen identified software defects.
- 11 A modular, clean backend route structure that keeps the codebase easy to extend with new endpoints without disturbing existing features.
- 12 A consistent, simple design system across all dashboard pages, reducing the analyst's cognitive load when moving between different views.

These advantages, considered together, directly address every limitation of the pre-existing manual approach identified in Section 3.2, and form the basis on which the detailed system design in Chapter 5 was carried out.

5. System Design

Chapter 4 proposed a six-stage pipeline and the six modules that implement it. This chapter descends one level further, presenting the concrete architecture, UML views, database schemas, and interface designs used to build those modules. Section 5.1 walks through the platform's High-Level Design (HLD) and Low-Level Design (LLD) workflows end-to-end. Section 5.2 presents the system through the standard set of UML views—structural (Entity-Relationship, Class, and Deployment) and behavioral (Use Case, Sequence, Activity, and Data-Flow diagrams)—which formalize what Section 5.1 describes narratively. Section 5.3 details the PostgreSQL and Neo4j schema designs, Section 5.4 describes the dashboard's visual design system, and Section 5.5 catalogues the screenshots of the interface.

The design decisions recorded in this chapter were not fixed up front; as described in Chapter 2, they evolved across the eight sprints—most visibly in the Week 2 migration from a post-centric to a post-free Neo4j schema, and the Week 6 restructuring of the main explorer page's structure to be compatible with the client-side router. This chapter presents the architecture in its final, as-built state. Where a design evolved materially over the course of development, that evolution is noted explicitly.

5.1 System Architecture Design

The platform's High-Level Design (HLD) defines the structural architecture: the analyst's browser loads the single-page dashboard and communicates with the backend web server over standard web protocols and live data streams. The web server coordinates with a temporary memory cache, which stores repeated read requests to reduce database load. Behind the cache sits PostgreSQL for structured profile and post data, and Neo4j for network relationship queries.

A separate, background section of the platform handles the scraping pipeline: the web server triggers a "start scraper" action, which adds a job to the queue. A background worker thread picks up this job and launches a browser session to gather timeline data from the target social network. The collected content passes through a parsing and rate-limiting phase (which extracts entities, classifies topics, and enforces human-like scroll delays) before being written in batches to PostgreSQL. Simultaneously, this pipeline broadcasts live preview and status events back to the web server, which relays updates to every active dashboard client. Independently of this live scraper loop, a graph transformer periodically reads the accumulated PostgreSQL dataset, exports it to text-based CSV format, and a graph loader runs import queries to bring the Neo4j graph database in sync.

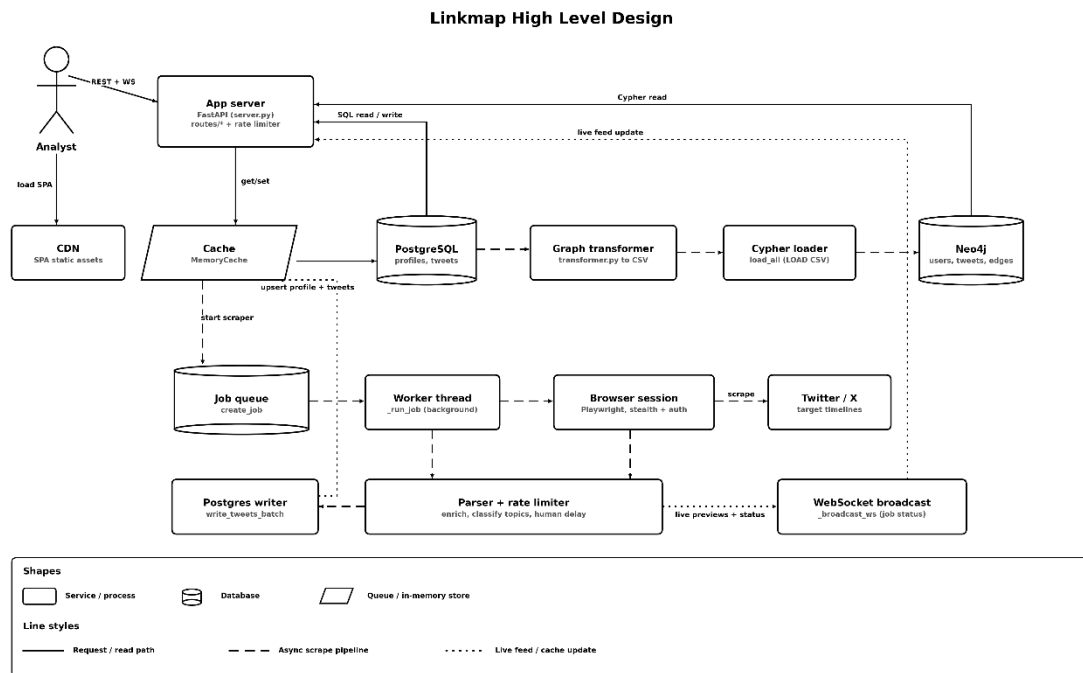


Figure 2 : High-Level Design (HLD) of the Linkmap OSINT Platform

The HLD separates the platform into three visual categories: the **request path** (handling web communication between the analyst and the web server), the **background collection pipeline** (managing the browser automation, parsing, and storage steps independently of standard page load requests), and the **live feed / cache flow** (handling the live status updates and memory caching that keep the interface responsive without constantly querying the relational database). Separating these three flows explicitly in the architecture is a deliberate design choice, ensuring that a reader can easily distinguish between synchronous user actions and background scraping tasks.

Descending to the Low-Level Design (LLD) details, we trace this data flow down to specific functions and files:

- 1. On the frontend:** Navigation functions (`initNav`, `showNav`, and `initPageTransitions`) manage the router in `nav.js`, which loads the page-specific controllers (such as `dashboard.js`, `scraper.js`, `network.js`, `hashtags.js`, `intelligence.js`, and others). Each controller calls backend API functions (like loading metrics, networks, mention tables, and coordination indicators) through a shared API helper utility in `utils.js`.
- 2. On the backend API:** Every incoming request first passes a rate-limiter check in the backend server. The request then proceeds to either the memory cache, the scraper manager (`create_job` or `stop_job` actions), or the graph database client.
- 3. During data collection:** The scraper manager initiates a background loop that starts a browser session (managed in `browser.py` and authenticated via `auth.py` using stored login cookies). The engine collects profile metadata and walks target timelines using the scraper logic in `tweet_scraper.py`, applying human-like scroll pacing to prevent

blocks. The parser (parser.py) extracts hashtags, user mentions, links, and keywords, classifying them by topic maps.

4. **During storage and sync:** The parsed batches are written to the relational database in postgres_writer.py (via write_profile and write_tweets_batch functions). Once collection completes, the graph transformer (transformer.py) runs its transformation script (transform) to output node and edge CSV files. The graph loader (cypher_loader.py) then runs its import pipeline (load_all) against the Neo4j database.
5. **Real-time updates:** Finally, the scraper manager broadcasts a completion signal over the WebSocket endpoint to all active frontend clients, triggering interface functions (updateLiveFeed and updatePreviewCards) to update the dashboard tables and graphs in real time.

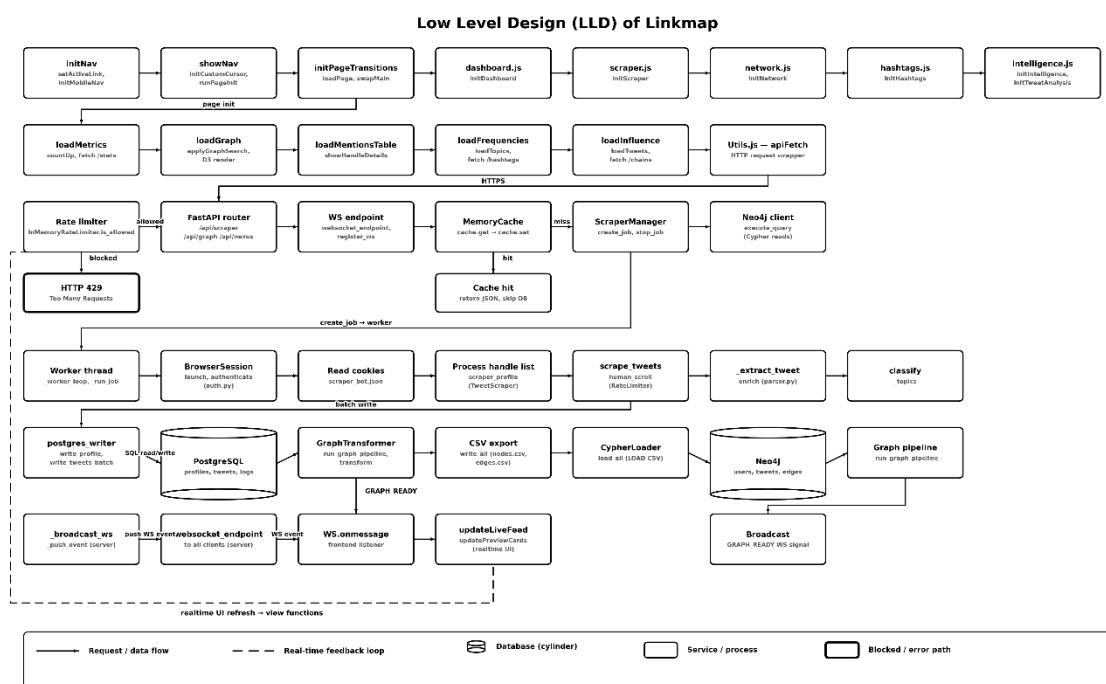


Figure 3 : Low-Level Design (LLD) of the Linkmap OSINT Platform

Where the HLD explains the major components and the flow of data between them, the LLD identifies which specific function, in which file, is responsible for each step. The low-level architecture is organized as eight horizontal layers: frontend navigation, page routing, frontend display views, the backend API gateway, backend core logic, the browser scraper worker, data storage pipelines, and the WebSocket live communication layer. Reading the LLD from top to bottom reconstructs the exact execution path from a user's click in the browser down to a database write in PostgreSQL or Neo4j.

5.2 UML Diagrams

The High-Level and Low-Level Design descriptions in Section 5.1 detail the platform's actual data flow. This section complements that view with the standard set of Unified Modeling Language (UML) diagrams. These formalize the system from two perspectives: its static structure (including entity-relationship and class structures) and its dynamic behavior (including use case, activity, sequence, and data-flow interactions).

Each of the six diagrams in this section (Sections 5.2.1–5.2.6) is presented as a labeled placeholder together with a description of the elements it is intended to depict, drawn directly from the actual entities, classes, and workflows implemented in this project. The finished diagrams themselves are to be inserted into the corresponding placeholder boxes.

5.2.1 Use Case Diagram

The primary actor in the platform's use-case model is the OSINT Analyst, with a secondary System Administrator actor (the Data/Graph Engineer) covering configuration and system maintenance. As Figure 4 shows, the Analyst's use cases are grouped into three main areas inside the system boundary:

- 1. Data Collection & Ingestion:** Starting and stopping data collection runs, monitoring progress updates via WebSockets, configuring target handles, setting limits on posts collected, and viewing past run history.
- 2. Analysis & Exploration:** Reviewing dashboard metrics, examining account metrics, analyzing hashtag frequencies, grouping topics, exploring D3 network charts, using the graph explorer, monitoring coordinated campaign events, searching posts, and filtering the data explorer.
- 3. Reporting & Data Export:** Exporting PDF intelligence reports, filtering accounts by date range, and choosing target handles for reporting.

Internal system use cases (such as automated browser collection, parsing and enriching post text, storing results in PostgreSQL, building the Neo4j graph relationships, and broadcasting WebSocket alerts) represent backend pipeline steps triggered indirectly by the Analyst. These connect outward to external systems: the target social network, the relational database, and the graph database.

Linkmap OSINT Platform - Use Case Diagram

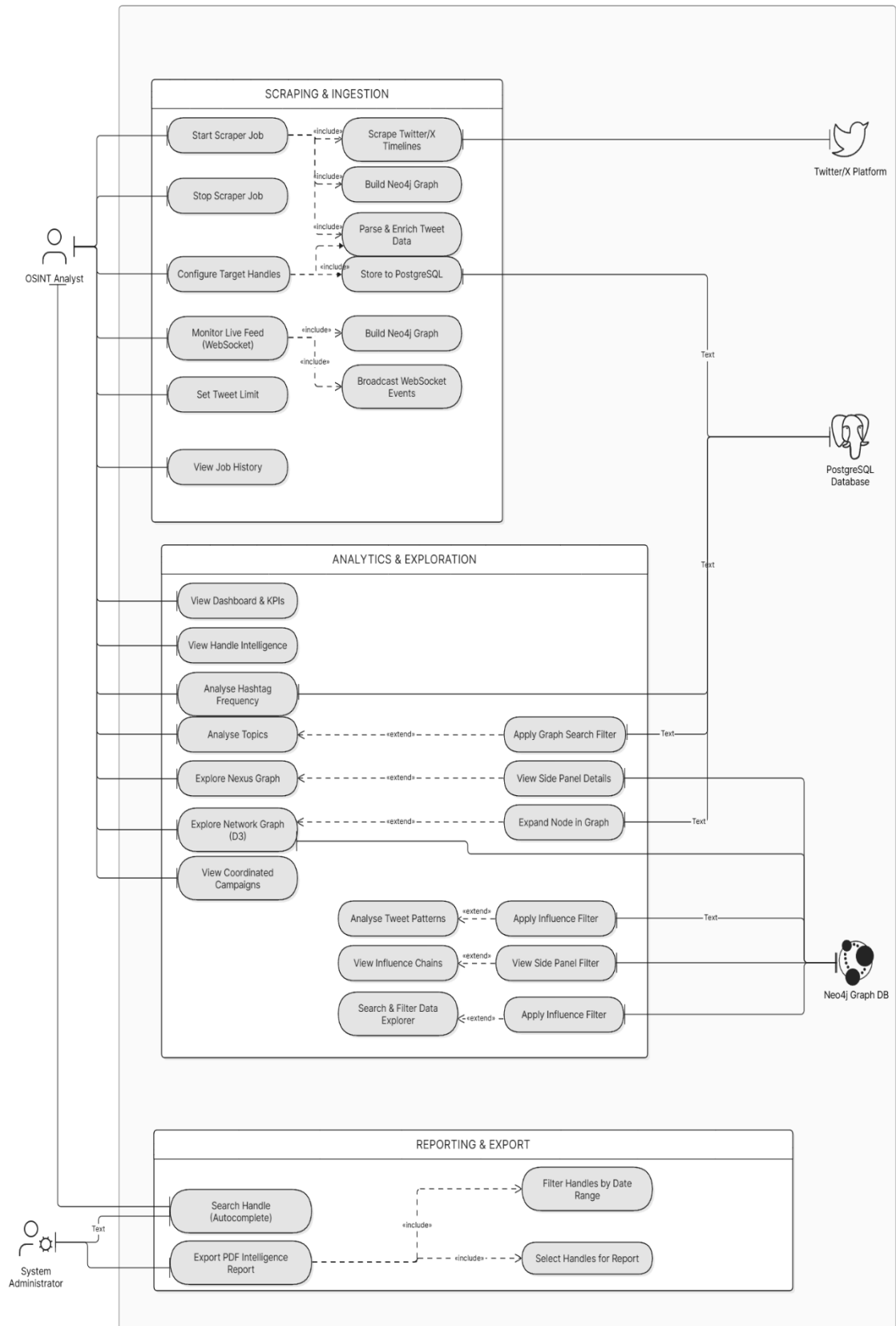


Figure 4 : Use Case Diagram

5.2.2 Activity Diagram

Figure 5 models the step-by-step workflow of a data collection job as a sequence of activities:

1. **Initialization:** The process starts by reading saved browser cookies and authenticating the automated browser session (retrying with a delay if authentication fails).
2. **Account Loop:** For each configured handle, the system applies a cool-down delay, loads the profile metadata, scrolls the account timeline at human-like intervals, and extracts and processes each post.
3. **Deduplication & Storage:** The workflow checks if a post has already been stored; duplicate posts are skipped, while new posts are saved in batches to PostgreSQL.
4. **Graph Update & Completion:** Once all target timelines are processed, the system triggers the graph pipeline to export database records and sync relationships into Neo4j. Finally, it broadcasts a completion alert to all connected dashboard pages.

Two critical decision points are defined in this flow: the duplicate check (which skips already-stored posts to prevent database bloat) and a session health check (which pauses the collection after 500 posts to mimic natural browsing patterns). Both safety checks are treated as mandatory steps in every collection run.

Linkmap Scraper Job — Activity Diagram

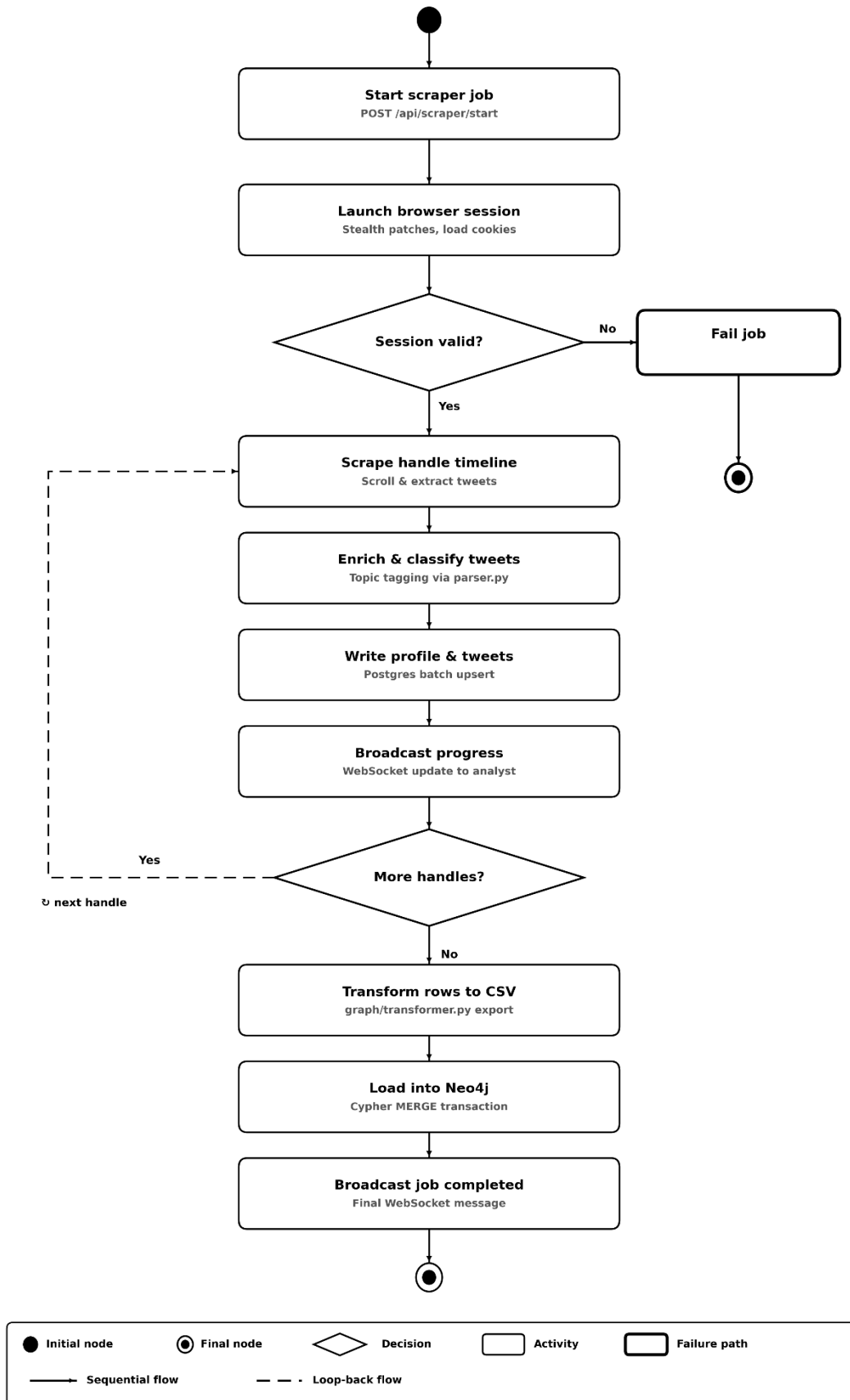


Figure 5 : Activity Diagram — Per-Handle Scrape Workflow

5.2.3 Sequence Diagram

The sequence formally modeled in Figure 6 is the platform's core scraper-job execution flow, tracing interaction steps across the system's components:

1. **Trigger:** The Analyst initiates a collection job from the browser, which sends a start request to the backend scraper manager.
2. **Scraper Execution:** The manager launches a browser session, navigates to target timelines, and loops through human-like scrolls to collect profile details and posts.
3. **Parsing & Storage:** Raw posts are passed to the parsing utility to extract entities and map narrative topics. The backend then runs database write functions to store the profiles and posts in PostgreSQL.
4. **Graph Sync:** Once collection finishes, the scraper manager calls the graph transformer, which queries PostgreSQL, builds node identifiers, and writes temporary CSV files. The Cypher loader is then triggered to import these CSV files into Neo4j.
5. **Broadcast:** The scraper manager broadcasts a final completed event to the frontend client over WebSockets to refresh the user dashboard in real time.

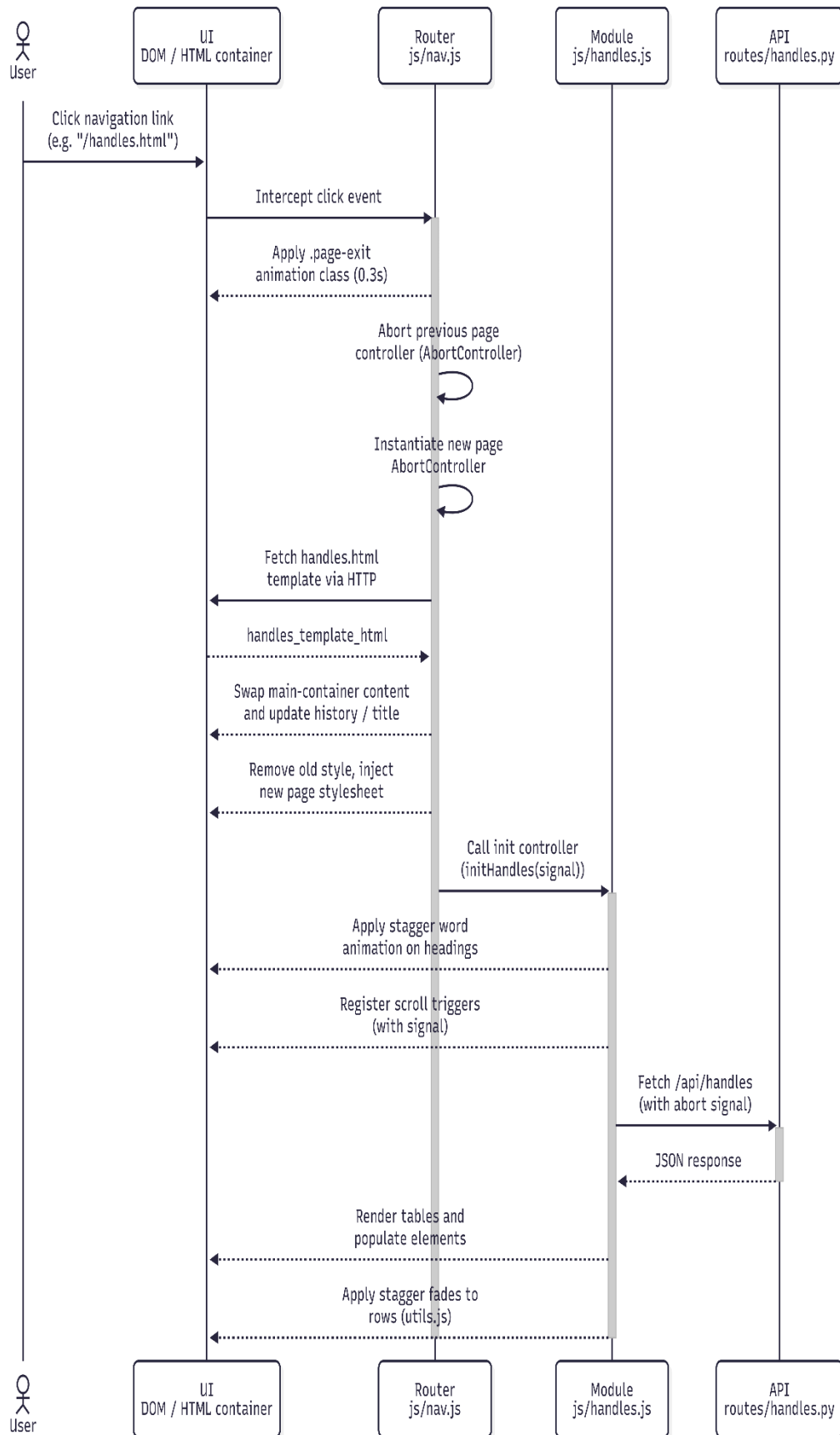


Figure 6 : Sequence Diagram — Triggering a Scrape Job

5.2.4 Class Diagram

Figure 7 centers on the backend's principal classes and documents their static structures and relationships:

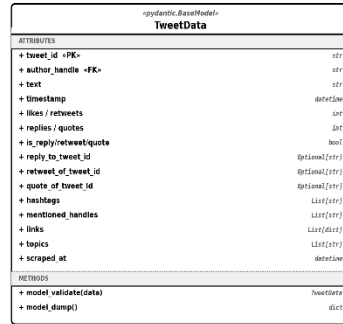
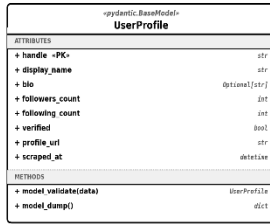
1. **Data Classes:** Simple data holders like the scraper job status object (tracking current handles, progress logs, and database sync status) and Pydantic models for profiles and posts that validate all collected data.
2. **Orchestrator Classes:** Single-instance managers like the scraper manager (coordinating start and stop commands), the graph transformer (preparing node and edge files), and the Cypher loader (executing graph import scripts).
3. **Infrastructure Clients:** Wrapper classes that manage connections to external resources, such as browser automation clients and connection pools for PostgreSQL and Neo4j databases.

Associations between these classes reflect system dependencies, ensuring that data is validated, stored, transformed, and loaded in a consistent, structured order.

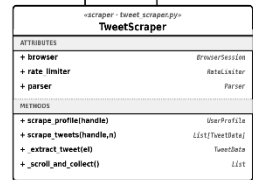
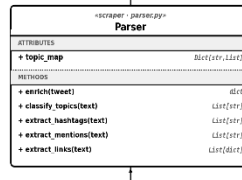
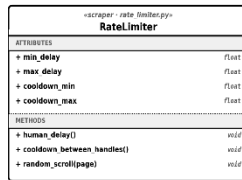
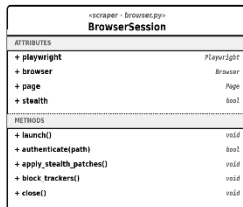
LINKMAP OSINT PLATFORM — CLASS DIAGRAM

Figure 5.5 | UML Class Notation | Attributes | Methods | Relationships

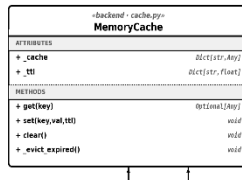
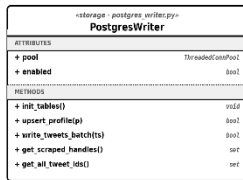
DATA MODELS (PYDANTIC) — STORAGE/SCHEMA.PY



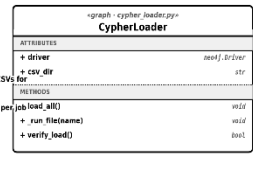
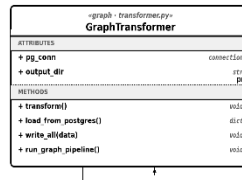
SCRAPER LAYER (PLAYWRIGHT) — SCRAPER/



STORAGE LAYER — STORAGE/ - BACKEND/CACHE.PY



GRAPH ETL LAYER — GRAPH/



INFRASTRUCTURE & API LAYER

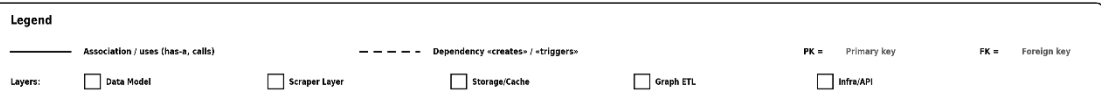
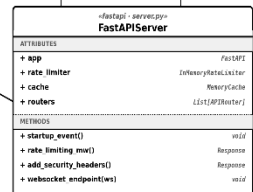
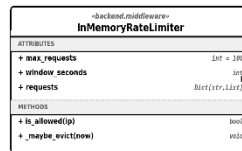
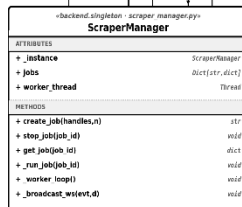
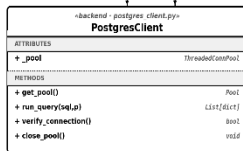


Figure 7 : Class Diagram

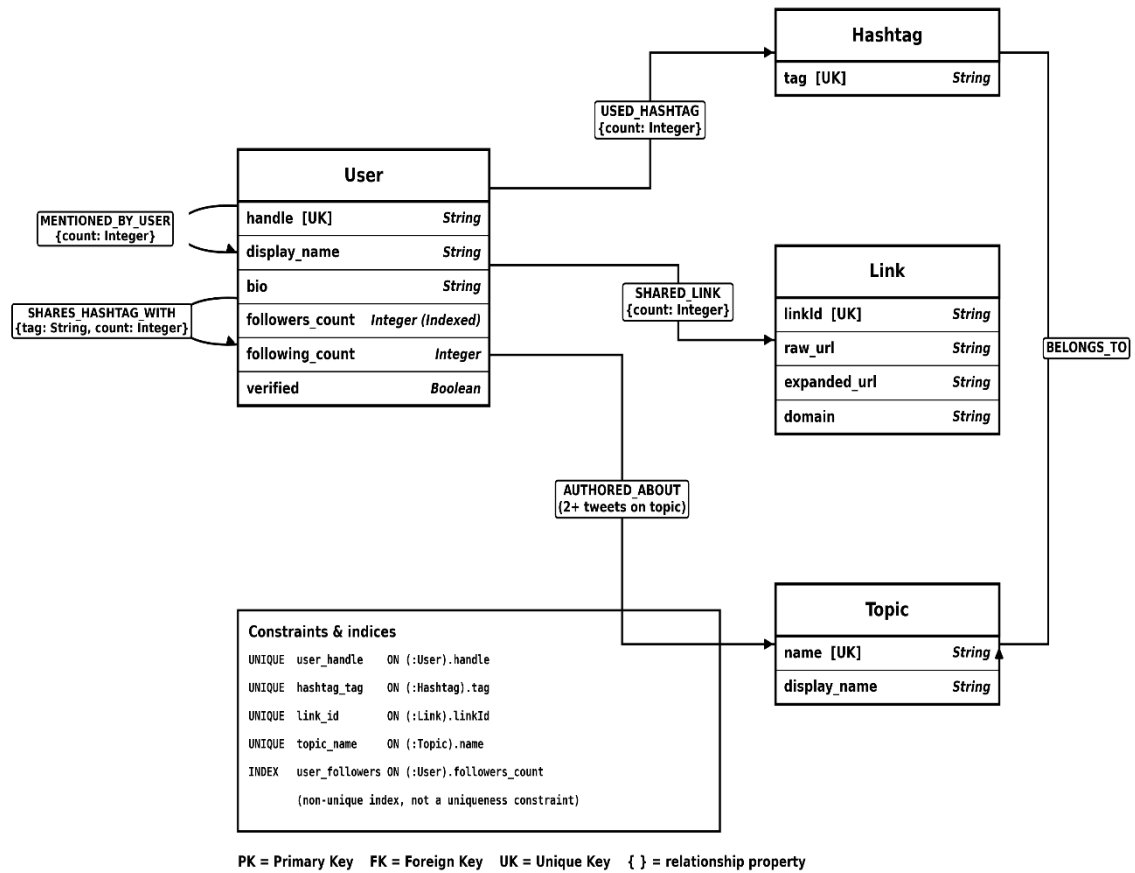
5.2.5 Entity-Relationship (E-R) Diagram

The Entity-Relationship diagram in Figure 8 documents the relational side of the platform's storage layer, representing the PostgreSQL table structures:

1. **Profiles Entity:** Stores account details using the handle as the unique primary key, along with display names, biography descriptions, follower counts, verified badges, and timestamps.
2. **Tweets Entity:** Stores post details using a unique post identifier as the primary key. It includes a foreign key pointing to the author's handle in the profiles table, establishing a one-to-many relationship (one profile authors many posts).

Columns like hashtags, mentions, links, and topic categories are defined clearly within the tables. The complementary graph schema in Neo4j (representing nodes and relationships) uses a different structure and is updated via a periodic sync script rather than direct writes; it is therefore detailed separately in Section 5.3.

Linkmap — Neo4j Schema (Graph)



Linkmap — PostgreSQL Schema (Relational)

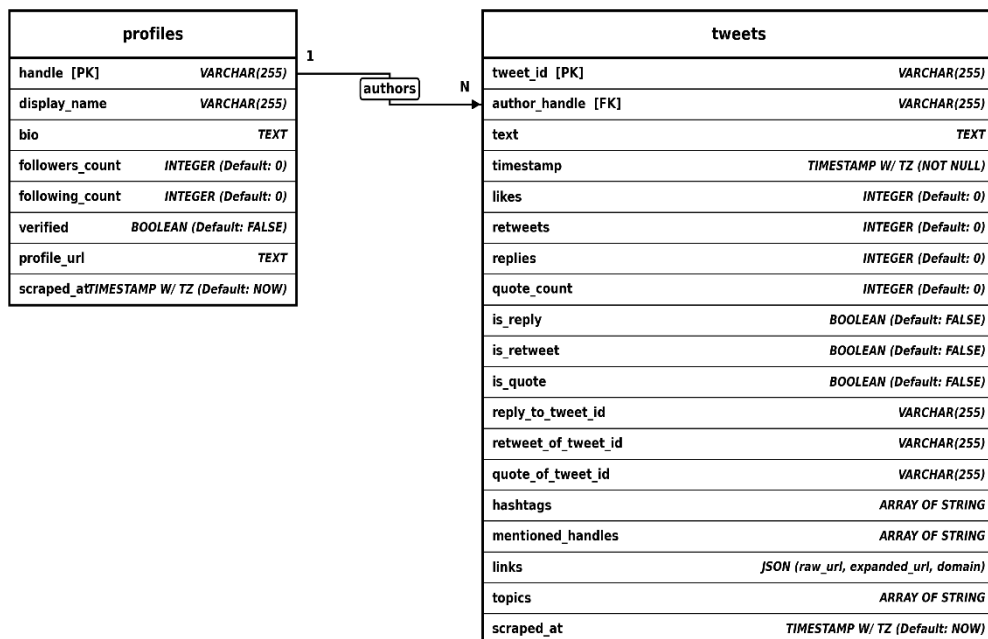


Figure 8 : Entity-Relationship (E-R) Diagram

5.2.6 DFD Diagram

The Data Flow Diagrams represent how information moves through the system at two levels of detail:

1. **Level 0 (Context Diagram):** As Figure 9 shows, the context diagram represents the platform as a single process interacting with two external entities: the Analyst and the target social network. Data flows include search filters and target handles sent to the social network, post data flowing back in, and visual dashboard views and PDF reports sent to the Analyst.

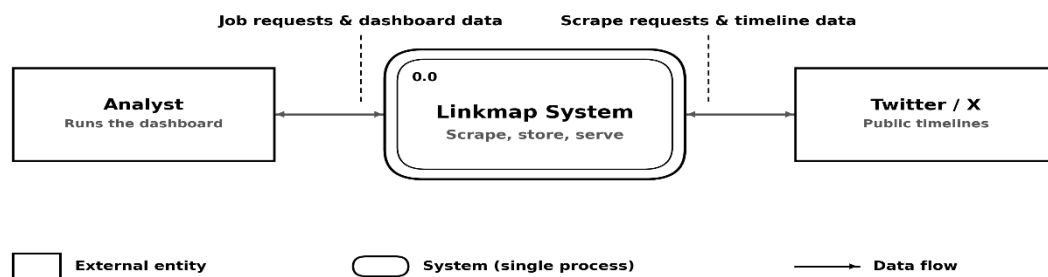


Figure 9 : DFD Level 0 — Context Diagram

2. **Level 1 (Module Diagram):** Decomposes the platform into its six functional modules—Scraper, Storage, Graph, Backend, Frontend, and Reporting. As Figure 10 shows, it lists the PostgreSQL and Neo4j databases as explicit data stores, mapping the exact paths data takes from browser collection to database storage, graph sync, API routing, and frontend rendering.

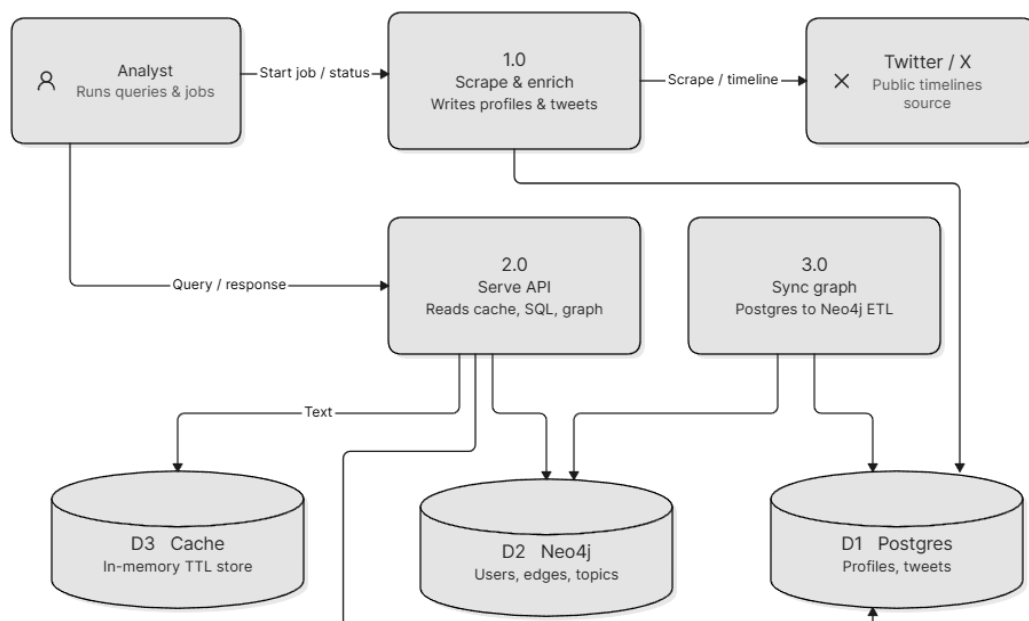


Figure 10: DFD Level 1 — Module Decomposition

5.3 Database Design

The platform's data layer is deliberately split across two purpose-fit stores rather than modeled in a single database. PostgreSQL is the system of record for all raw scraped data—profiles and posts—and is optimized for the filterable, paginated, keyword-searchable queries used throughout the search and explorer views. Neo4j is a derived, periodically rebuilt property graph optimized for the relationship traversals used throughout the network explorer views and campaign-detection queries. This section documents the design of both stores: table design and relationships (Section 5.3.1), the normalization reasoning applied to the relational schema (Section 5.3.2), and the database fields (Section 5.3.3).

A deliberate design decision, made during the Week 2 schema migration described in Chapter 2, was to keep Neo4j free of individual post nodes. Instead of modeling every post as a graph node, the graph transformer collapses post-level data into direct entity-to-entity edges (such as `USED_HASHTAG`, `SHARED_LINK`, `SHARES_HASHTAG_WITH`, and `MENTIONED_BY_USER`) between the User, Hashtag, Link, and Topic nodes. This keeps the graph database compact and faster to query, while preserving every relationship needed for network analysis and campaign detection.

5.3.1 Table Design & Relationships

Figure 11 shows the PostgreSQL schema visually—featuring the same `PROFILES` and `TWEETS` entities as Figure 8, reproduced here alongside the relational design discussion—and Figure 12 shows the Neo4j graph schema: the User, Hashtag, Link, and Topic node labels and the relationships connecting them. The details of the two databases are summarized below.

TABLE 9 : POSTGRESQL TABLE DESIGN

Table	Key Columns	Relationship
profiles	'handle' (Primary Key)	Stores target account metadata. Has a one-to-many (1:N) relationship pointing to the 'tweets' table.
tweets	'tweet_id' (Primary Key), 'author_handle' (Foreign Key)	Stores collected post metadata. References 'profiles.handle' to link posts back to their authors.

Linkmap — PostgreSQL Schema (Relational)

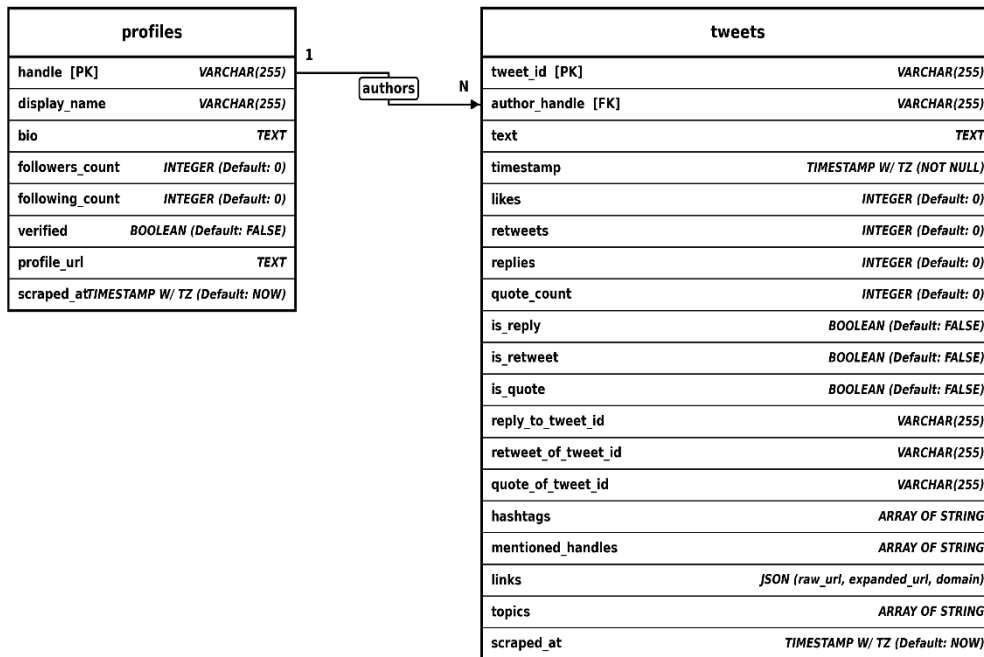


Figure 11 : PostgresSQL Schema

TABLE 10 : NEO4J NODE & RELATIONSHIP DESIGN

Node Label	Key Outgoing Relationships	Target Nodes	Purpose
`User`	`USED_HASHTAG`	`Hashtag`	Maps which accounts used which hashtags.
	`SHARED_LINK`	`Link`	Maps which accounts shared which URLs.
	`SHARES_HASHTAG_WITH`	`User`	Maps accounts sharing identical hashtags.
	`AUTHORED_ABOUT`	`Topic`	Links accounts to narrative topics.
	`MENTIONED_BY_USER`	`User`	Maps direct account-to-account mentions.
`Hashtag`	`BELONGS_TO`	`Topic`	Associates hashtags with topic groups.
`Link`	None (Incoming only)	None	Node representing external URLs.
`Topic`	None (Incoming only)	None	Node representing narrative categories.

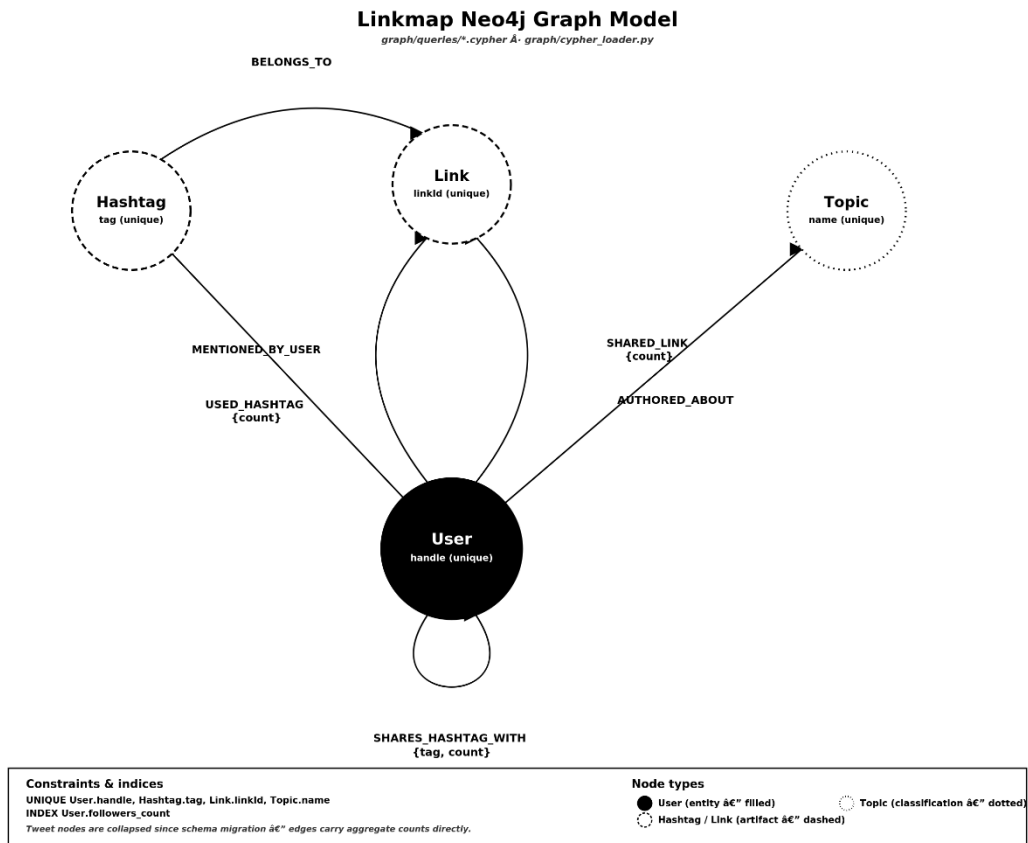


Figure 12 : Neo4j Graph Model Schema

5.3.2 Normalization

The PostgreSQL database schema was normalized to Third Normal Form (3NF) with two deliberate, documented exceptions to optimize query performance.

The profiles table satisfies First Normal Form (1NF) as every column holds a single atomic value. It satisfies Second Normal Form (2NF) since its primary key (handle) is a single column and all other columns depend directly on it. It satisfies Third Normal Form (3NF) because no non-key column depends on another non-key column (for instance, follower count does not determine the user biography, or vice versa).

The tweets table contains two denormalizations to avoid expensive query joins:

1. **Array Columns:** The hashtags, mentioned_handles, and topics columns are stored as native array types (TEXT[]) rather than splitting them into separate lookup tables.
2. **JSON Column:** The links column is stored as a single JSONB document rather than a normalized links table.

This design choice was made to speed up dashboard queries. A fully normalized design would require joining across multiple tables to reconstruct a single post's details (hashtags, mentions, links, and topics) on every search or dashboard view. PostgreSQL's native array and JSONB indices support rapid containment queries directly, giving the platform the performance of a flat structure with the filtering capability of a relational model.

This denormalization is safe because these columns are write-once, derived-at-parse-time values that are never modified after a post is first stored. The profiles table, by contrast, is updated frequently (such as follower counts changing on re-scrapes) and was therefore kept fully normalized to prevent anomalies.

The Neo4j graph database does not use relational normalization rules. Instead, the design prevents duplicating property values across multiple relationship types where a single edge property or link is sufficient.

5.3.3 Data Dictionary

The details of the PostgreSQL columns and data types for both core tables are outlined below:

TABLE 11 : POSTGRESQL DATA DICTIONARY (PROFILES & TWEETS)

Table	Column Name	Data Type	Description / Constraints
`profiles`	`handle`	`VARCHAR`	Primary Key. Unique handle of the account (e.g., @name).
	`display_name`	`TEXT`	Public display name of the user profile.
	`bio`	`TEXT`	Profile biography text.
	`followers_count`	`INTEGER`	Number of followers at the time of collection.
	`following_count`	`INTEGER`	Number of followed accounts at the time of collection.
	`verified`	`BOOLEAN`	Verified badge status of the account.
	`profile_url`	`TEXT`	Canonical web URL of the profile.
	`scraped_at`	`TIMESTAMPTZ`	Timestamp when the profile row was written.
`tweets`	`tweet_id`	`VARCHAR`	Primary Key. Unique identifier of the post.
	`author_handle`	`VARCHAR`	Foreign Key. References `profiles.handle`.
	`text`	`TEXT`	Full text content of the post.
	`timestamp`	`TIMESTAMPTZ`	Posting date and time.

Table	Column Name	Data Type	Description / Constraints
	`likes`	`INTEGER`	Number of likes.
	`retweets`	`INTEGER`	Number of retweets/reposts.
	`replies`	`INTEGER`	Number of replies.
	`quote_count`	`INTEGER`	Number of quote reposts.
	`is_reply`	`BOOLEAN`	True if the post is a reply.
	`is_retweet`	`BOOLEAN`	True if the post is a repost/retweet.
	`is_quote`	`BOOLEAN`	True if the post is a quote.
	`hashtags`	`TEXT[]`	Array of extracted hashtags.
	`mentioned_handles`	`TEXT[]`	Array of mentioned user handles.
	`topics`	`TEXT[]`	Array of classified topic tags.
	`links`	`JSONB`	Structured object containing expanded external links.
	`scraped_at`	`TIMESTAMPTZ`	Timestamp when the post row was written.

5.4 GUI Design

The dashboard's visual interface follows a consistent design system applied across all pages. The colour palette uses an off-white canvas background (#F0EDE8) with solid black text, sharp borders, no drop shadows, and thin dividing lines. This choice establishes a clean, print-inspired visual style rather than a rounded web dashboard look.

Typography is governed by three specific font families:

1. **Serif Font (Playfair Display):** Used for large page titles to give a distinct editorial look.
2. **Monospace Font (JetBrains Mono):** Used for numbers, stats, and data values to ensure clean alignment and readability.
3. **Sans-Serif Font (Inter):** Used for standard body text, buttons, and input labels.

This typography is applied universally via central stylesheets, making it simple to add new dashboard pages without introducing new styling rules.

The network and graph explorer pages switch to a dark canvas background (#0F172A) with a subtle dot-grid. This dark theme helps visualize network connections clearly and uses five node colors:

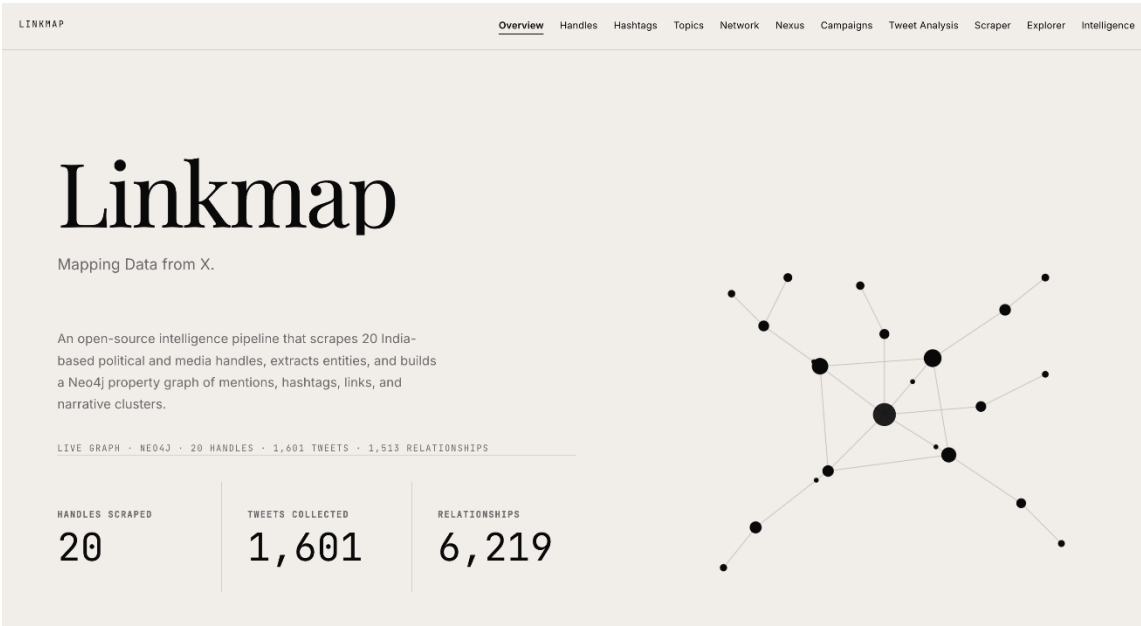
- 1. **White:** User accounts
- 2. **Cyan:** Hashtags
- 3. **Purple:** User mentions
- 4. **Orange:** Web links
- 5. **Green:** Topics

This color coding helps analysts immediately identify node types. The rest of the interface uses skeleton loading animations while data loads from the API backend, along with clear alerts and notifications. Filter groups use uniform dropdown menus and search inputs across all views, ensuring a predictable user experience.

5.5 Screenshots

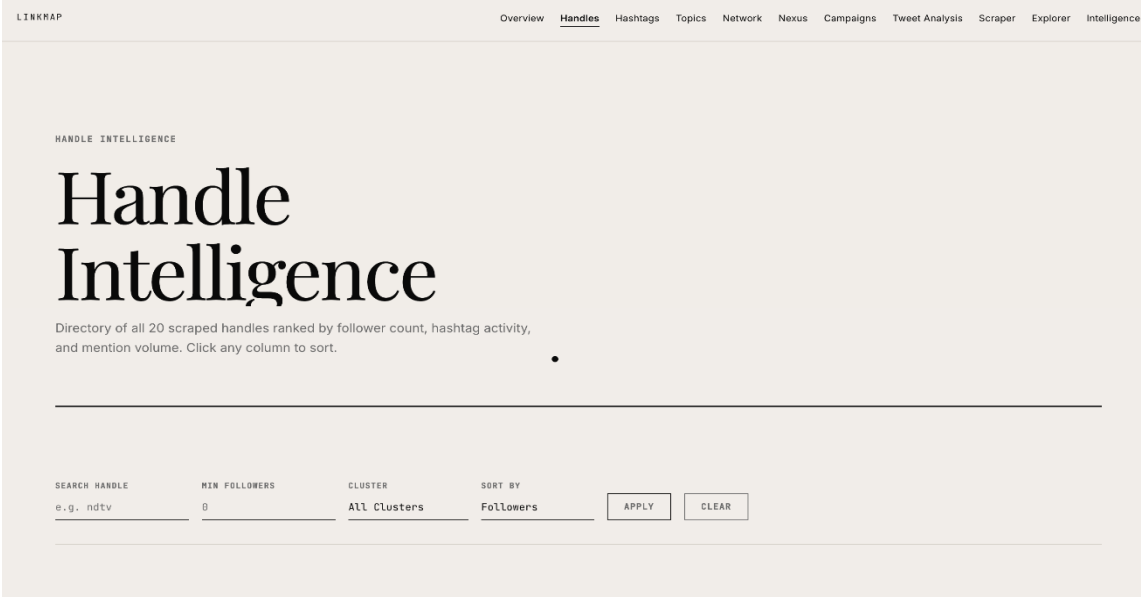
This section catalogues the screenshots to be inserted for each of the twelve dashboard pages and the generated PDF report, illustrating the visual interface design described in Section 5.4 as it appears in the running application.

Screenshot 1: Overview Dashboard The landing page opens with the Linkmap wordmark, a one-line project description, and a live stats strip reporting 20 accounts scraped, 1,601 posts collected, and 6,219 relationship connections, alongside an animated preview of the underlying network.



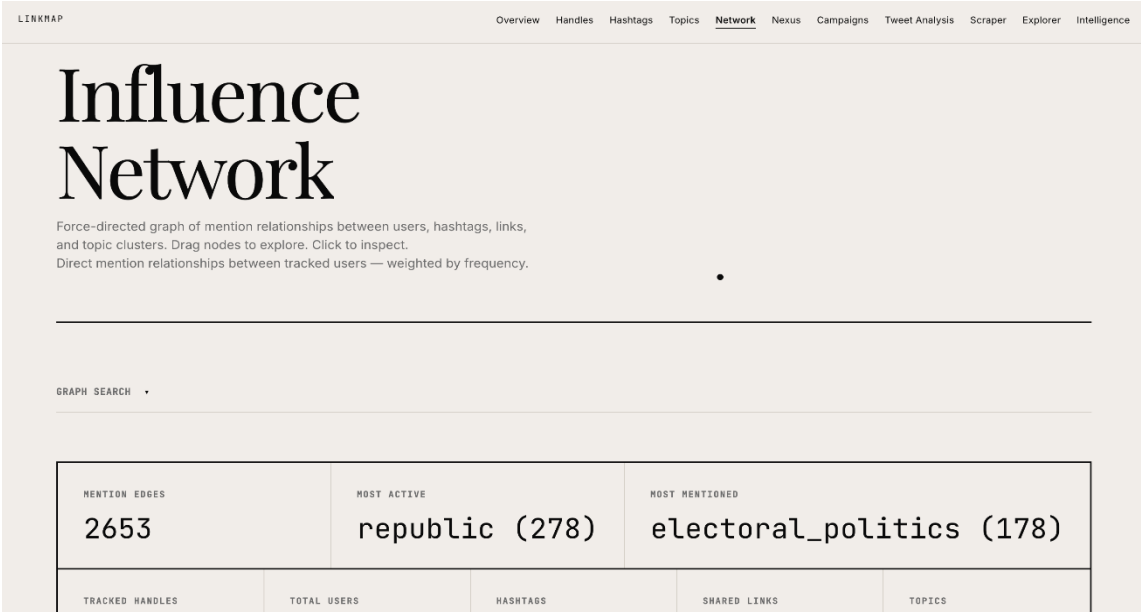
SCREENSHOT 1 : OVERVIEW — LANDING PAGE WITH LIVE STAT COUNTERS AND ANIMATED NETWORK PREVIEW

Screenshot 2: Handle Intelligence The Handle Intelligence page presents a directory of all 20 scraped handles beneath a filter bar for searching by handle, minimum follower count, cluster category, and sort order, styled within the platform's editorial page design.



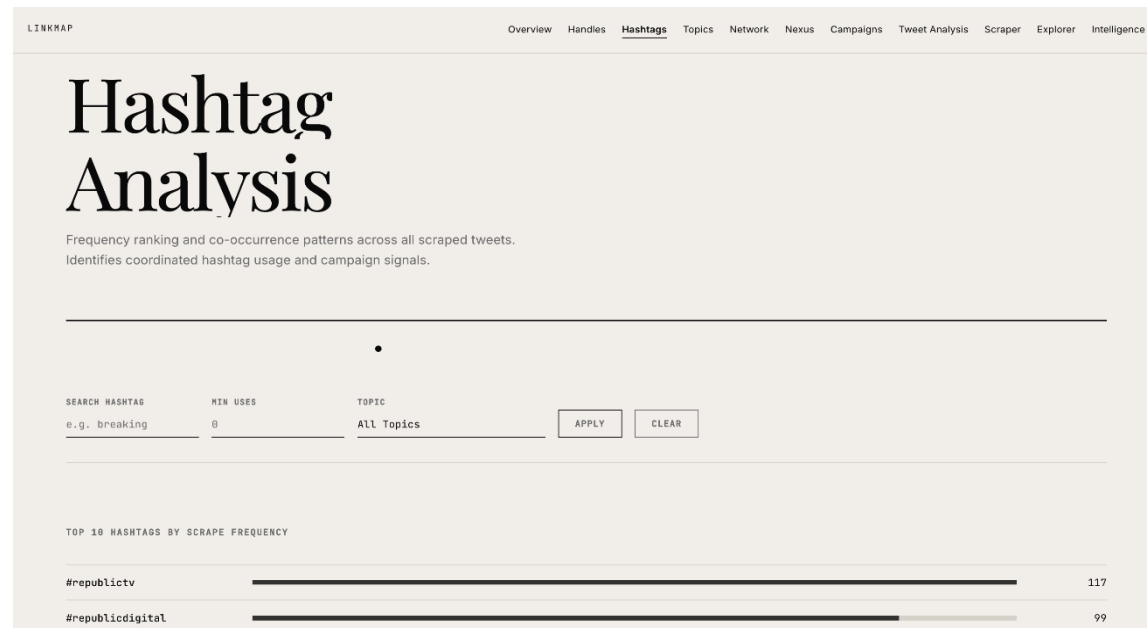
SCREENSHOT 2 : HANDLE INTELLIGENCE — FILTERABLE HANDLE DIRECTORY WITH CLUSTER BARS

Screenshot 3: Influence Network The Influence Network page renders the mention graph as an interactive network chart, with a live stats bar reporting 2,653 mention connections, showing the most active account and the most-mentioned narrative topic, alongside a collapsible search panel.



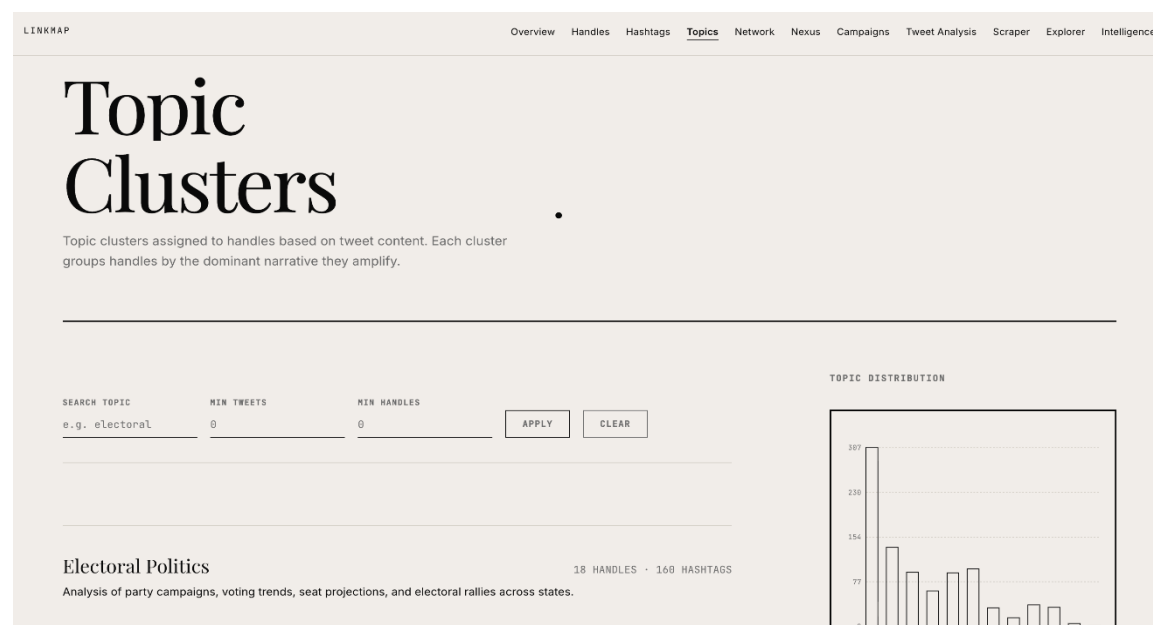
SCREENSHOT 3 : INFLUENCE NETWORK — D3.JS FORCE-DIRECTED GRAPH WITH GRAPH SEARCH PANEL

Screenshot 4: Hashtag Analysis The Hashtag Analysis page combines a search and filter bar (supporting hashtag, minimum uses, and topic parameters) with a horizontal frequency ranking of the top ten hashtags observed in the dataset.



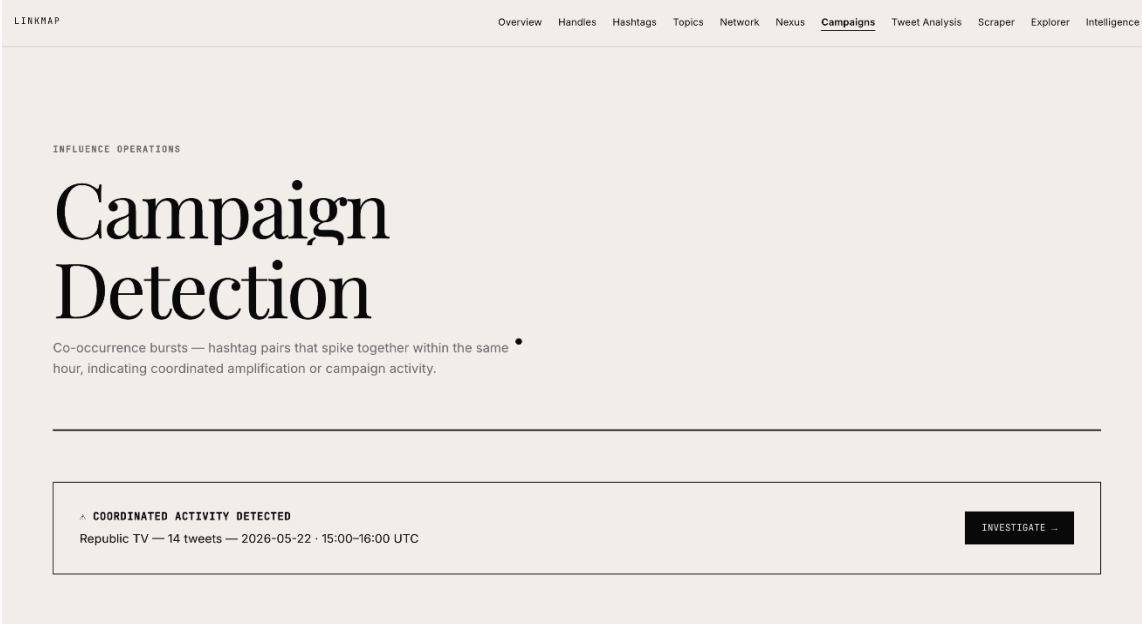
SCREENSHOT 4 : HASHTAG ANALYSIS — FREQUENCY BARS, CO-OCCURRENCE TABLE, BURST ALERT

Screenshot 5: Topic Clusters The Topic Clusters page lists each narrative cluster with its account and hashtag counts, alongside a vertical topic-distribution chart displayed in the sidebar panel.



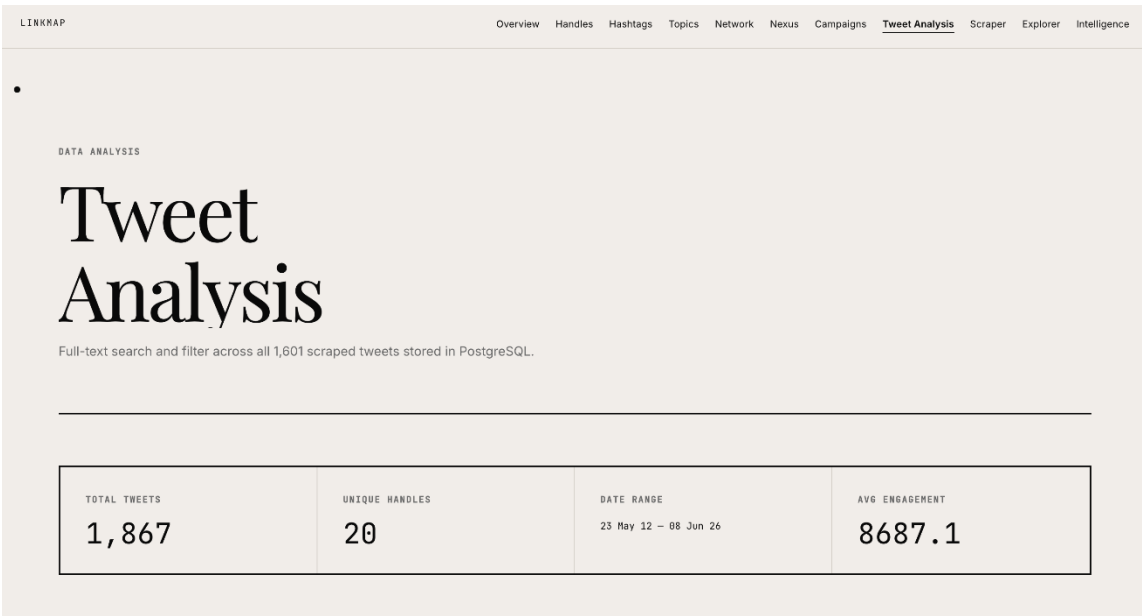
SCREENSHOT 5 : TOPIC CLUSTERS — EXPANDABLE TOPIC ROWS WITH DISTRIBUTION CHART

Screenshot 6: Campaign Detection The Campaign Detection page opens with an alert banner flagging coordinated communication pattern, showing target accounts posting multiple times within a short time window, with an option to investigate the underlying activity details.



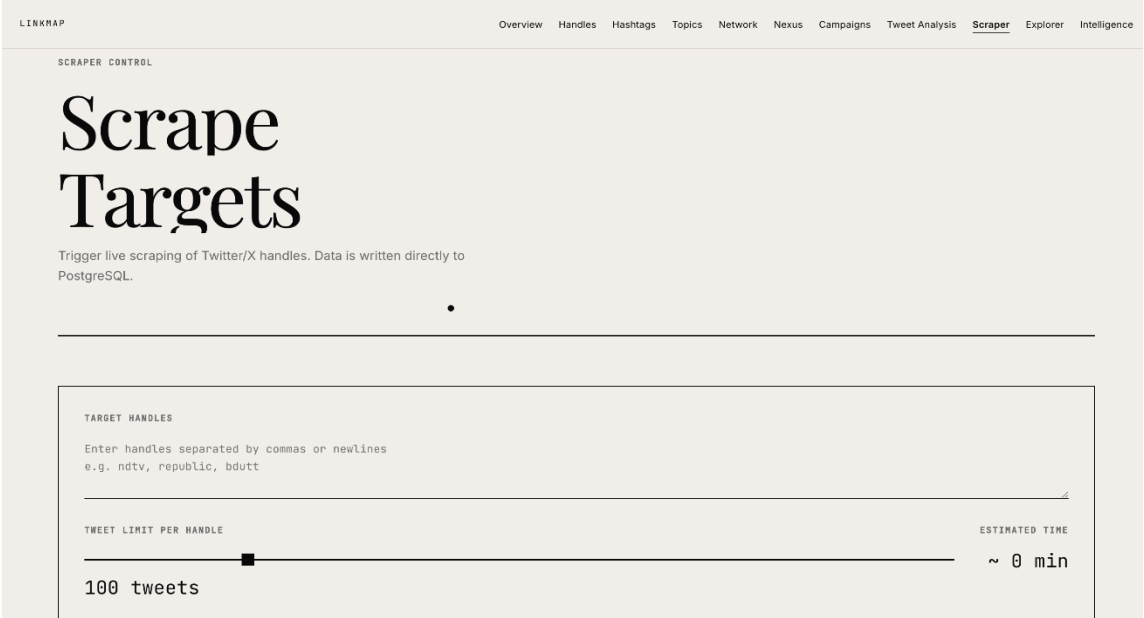
SCREENSHOT 6 : CAMPAIGN DETECTION — COORDINATED-ACTIVITY ALERT AND RADIAL HUB

Screenshot 7: Tweet Analysis The Tweet Analysis page displays search coverage across all collected posts, listing total posts, date ranges, and average engagement statistics above a six-filter search table.



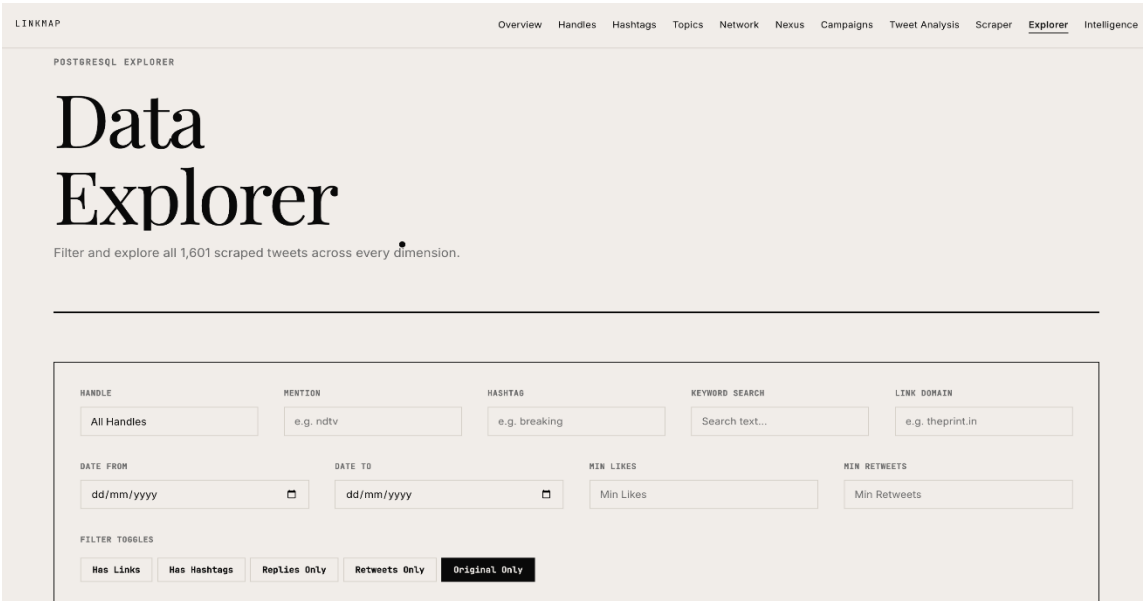
SCREENSHOT 7 : TWEET ANALYSIS — SIX-FILTER SEARCH TABLE WITH SVG CHARTS

Screenshot 8: Scraper Control Panel The Scraper Control Panel view shows the target account text input area, a collection limit slider, and a live estimated duration readout, ready to start a job whose progress updates in real time.



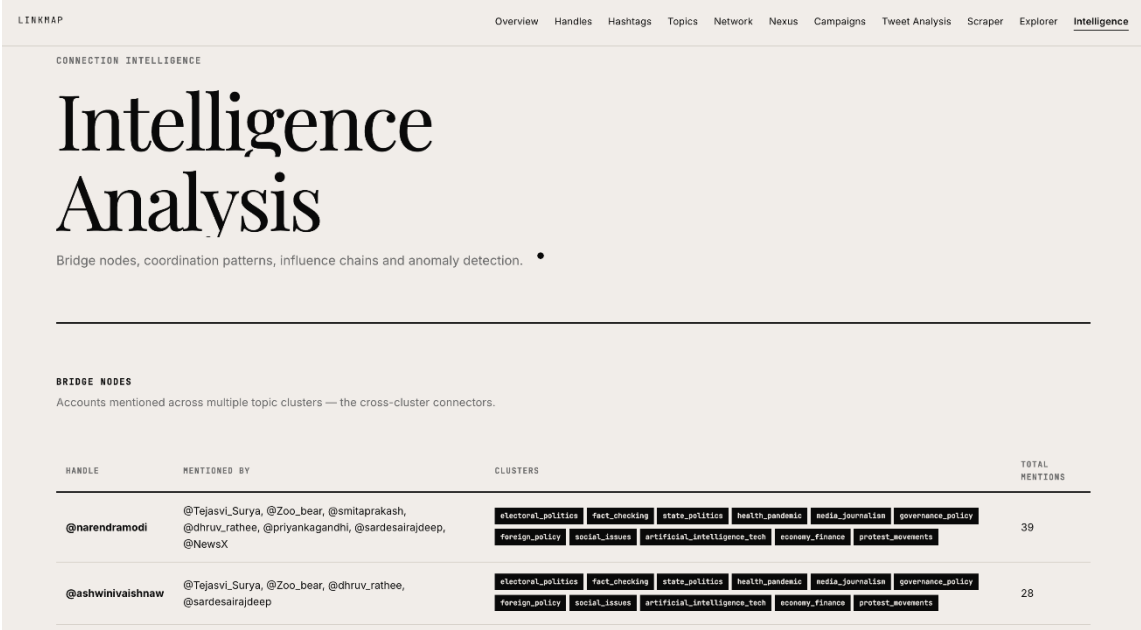
SCREENSHOT 8 : SCRAPER CONTROL PANEL LIVE WEBSOCKET PROGRESS FEED AND JOB HISTORY

Screenshot 9: Data Explorer The Data Explorer displays multiple search filters at once (such as handle, mention, hashtag, keyword search, web domain, and date range), plus toggle buttons for links, hashtags, and post types, to filter across the entire post dataset.



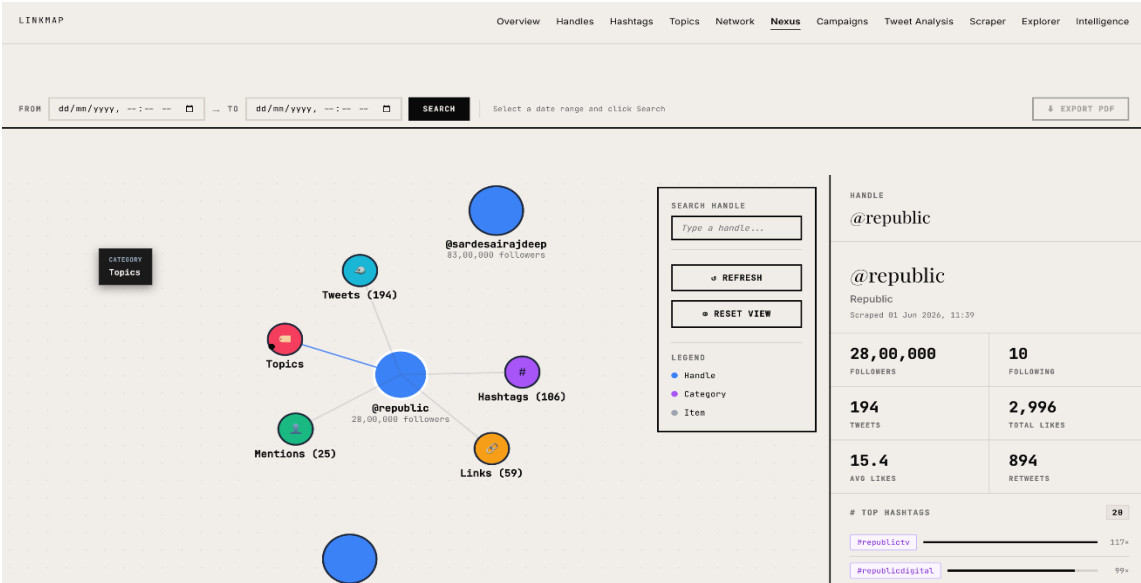
SCREENSHOT 9 : DATA EXPLORER — SEVEN-FILTER POSTGRESQL SEARCH WITH CSV EXPORT

Screenshot 10: Intelligence Analysis The Intelligence page's Bridge Nodes view identifies accounts mentioned across multiple narrative clusters, highlighting connectors that an analyst would otherwise have to search for manually.



SCREENSHOT 10 : INTELLIGENCE — BRIDGE NODES, CHAINS, CLUSTERS AND ANOMALY FLAGS

Screenshot 11: Nexus Graph Explorer The Nexus explorer shows target accounts expanded into categories like posts, topics, mentions, links, and hashtags, with a split side panel reporting profile metrics and top hashtags.



SCREENSHOT 11 : NEXUS GRAPH EXPLORER — SPLIT-PANEL GRAPH WITH AUTOCOMPLETE AND SIDE PANEL

Screenshot 12: Automated PDF Report The generated PDF opens with a cover page listing the reporting timeframe, selected accounts, and timestamps, followed by detailed profile summaries, hashtags, mentions, and top posts.

LINKMAP NEXUS

OSINT Intelligence Report

Period: 2026-06-03T01:28 → 2026-07-07T01:28

Handles: @ndtv, @timesofindia

Generated: 09 Jul 2026, 01:28 UTC

1. HANDLE PROFILES

@ndtv

NDTV

Followers	Following	Tweets	Total Likes	Total RTs	Avg Likes
17,200,000	20	63	279	69	4.4

#	Hashtag	x
1	#breaking	9
2	#breakingground	6
3	#watch	3
4	#forbes	1
5	#ekbharatshreshthabharat	1
6	#friendship	1
7	#globalprofile	1
8	#friendsofindia	1

#	Mention	x
1	@vedikas	6
2	@divyawadhwa	5
3	@tejshreethought	4
4	@pameshwarbawa	2
5	@dpkbopanna	2
6	@aishvaryain	2
7	@shivaroor	2
8	@gauried	1

Topics:

Media & Journalism (47) · Electoral Politics (8) · State Politics (3) · Foreign Policy (2) · Governance & Policy (2) · Health & Pandemic (2) · Digital Rights & Policy (1) · Artificial Intelligence & Tech (1)

SCREENSHOT 12 : AUTOMATED PDF REPORT — SAMPLE GENERATED INTELLIGENCE REPORT

6 Implementation Planning

With the architecture and schemas of Chapter 5 in place, this chapter describes how that design was actually turned into running software. Implementation was not a single monolithic phase but was spread across the eight sprints outlined in Section 2.2.2, with each sprint implementing a specific slice of the architecture described in Chapter 5:

1. **Weeks 1–2** implemented the Scraper, Storage, and Graph modules, alongside the first version of the Backend module.
2. **Week 3** implemented the Scraper Control Panel, Data Explorer, and Intelligence pages, while also hardening the Backend module against the security constraints described in Section 3.7.5.
3. **Week 4** implemented the interactive graph explorer page.
4. **Weeks 5 and 7** implemented the Reporting module.

This chapter is organized around the practicalities of that implementation process rather than around individual features: Section 6.1 describes the development environment used to build and run the system; Section 6.2 describes the specific developer-facing tools used day-to-day (as distinct from the runtime technology stack already listed in Section 1.5); and Section 6.3 records the coding standards the team followed to keep a three-person, multi-module codebase consistent and maintainable across the eight weeks of continuous change.

A recurring implementation theme, visible throughout the weekly milestones in Section 2.2.2, is that each new feature was built to integrate with the existing shared infrastructure—such as the connection-pool pattern established in Week 2, the rate-limiter and caching middleware established in Weeks 2 and 3, and the WebSocket broadcast patterns established in Week 3—rather than introducing parallel, feature-specific mechanisms. This discipline is one of the reasons the Week 3 audit (described in Chapter 7) found several categories of bugs (such as database query risks or memory leaks) that, once fixed in the shared patterns, were simultaneously fixed across every feature that used them.

Implementation planning also had to account for the fact that three people were modifying a shared, tightly coupled codebase simultaneously. Rather than adopt a complex branching strategy, the team relied on the natural separation provided by the three functional roles (Section 2.2.3): because the Data Engineer's changes were concentrated in the scraper and storage folders, the Graph Analyst's in the graph and Cypher query files, and the OSINT Analyst's in the frontend templates, most days produced few or no overlapping file changes. Where changes did overlap—most often at the boundary between backend routes and the frontend controllers that consumed them—the informal team agreement described in Section 2.2.4 (requiring advance notice of route changes) substituted for a more formal code gate.

6.1 Implementation Environment

All development was carried out on standard developer workstations meeting the specifications in Section 3.6, running a Python 3.11 virtual environment with dependencies managed via standard requirements configurations, including automated browser binaries. Each team member ran a full local copy of the stack—the relational database server, the local graph database instance, and the backend server—allowing independent development and testing without conflicts.

The project's folder structure was established in Week 1 as a modular layout (consisting of folders for scraping, storage, graph scripts, configuration files, and utilities) and was restructured once, in Week 4, to separate the backend API scripts from the static frontend templates. This required updating relative import paths throughout the server code but produced a cleaner separation between server-side and client-side code going forward.

Configuration was centralized in a small number of JSON files:

1. **settings.json** for timing parameters and folder paths.
2. **handles.json** for the twenty target accounts.
3. **topic_map.json** for the keyword mapping used by the topic extraction pipeline.

This allowed the target account list and topic keywords to be changed without touching source code, which directly supports the system's extensibility.

Environment variables, loaded via local environment configuration files, held all secrets and toggles—including social network credentials, database connection strings, and database activation flags used during early development. A Week 3 defect (described in Chapter 8) was traced specifically to a database flag not being loaded correctly inside the scraper manager's module imports, which silently dropped scraped posts rather than raising a visible error—a reminder that even a simple environment setup carries genuine implementation risk if loading sequences are not handled carefully.

Logging was treated as part of the implementation environment rather than an afterthought. Every module used a shared, file-rotation logger configured to write to local files with automatic cleanup. This proved essential during the Week 1 scraper debugging sessions, where the only way to distinguish a genuine rate-limit response from a broken web element was to inspect the detailed connection logs captured during a live data collection run.

6.2 Tools & Technologies Used

Beyond the runtime technology stack listed in Section 1.5, the following developer-facing tools were used throughout implementation:

1. **Git:** Used for version control, with the repository initialized in Week 5 alongside a structured ignore configuration covering virtual environments, local database folders, log archives, and credential files.
2. **Neo4j Developer Tools:** Used for writing, testing, and visually verifying Cypher queries during graph database schema development.
3. **Standard Code Editor:** A standard code editor with language support for Python and JavaScript, used for all backend routes, browser scraper scripts, and frontend dashboard views.
4. **Logging Library:** A rotating console and file logging utility, used extensively during the debugging of Week 1 browser scraper elements and the Week 3 security review.
5. **Browser Developer Tools:** The web browser console and network inspectors were used to verify real-time WebSocket communication flows and troubleshoot frontend dashboard layouts.
6. **Verification Scripts:** Standard compilation and search utilities were used in Week 7 as a lightweight static code check to verify syntax when live database instances were not running.

Deliberately absent from this list is any dedicated automated test runner, container orchestration tool, or continuous-integration service. This was a conscious scoping decision given the eight-week internship timeframe rather than an oversight, and is revisited explicitly as a recommended addition in Chapter 8's discussion of future scope.

6.3 Coding Standards Followed

The following coding standards were adopted at the start of the project and enforced informally through internal reviews at each sprint boundary:

1. **Python Code Style:** Python code follows PEP 8 style conventions, using descriptive function names, clear variable names, and introductory docstrings for every non-trivial script file.
2. **Parameterized SQL Queries:** All SQL queries are written using parameterized inputs with standard placeholder formats. String concatenation or f-string SQL queries are strictly disallowed—a rule introduced formally after the Week 3 audit identified SQL injection risks in two backend route files.
3. **Feature-Specific Backend Routes:** Every backend API route file corresponds to exactly one feature area (such as handles, hashtags, topics, campaigns, and network views), keeping the modular backend organization established in Week 2 consistent as the backend expanded.
4. **Database Connection Pooling:** All PostgreSQL database access is routed through a shared connection pool manager rather than creating new database connection instances on demand. This convention was introduced in Week 2 and retrofitted onto the command-line scraping pipeline in Week 3 to prevent connection build-up.
5. **Modular Frontend JavaScript:** Frontend scripts are written using JavaScript modules, separating controllers per page and sharing common utility scripts (for API requests and navigation rules) to avoid code duplication.

6. **Centralized Styling Customizations:** All shared CSS definitions reside in central component stylesheets instead of inline definitions, ensuring that visual updates (such as the filter bar styles developed in Week 3) propagate automatically to every dashboard view.
7. **Structured Version Commits:** Code commits and internal notes reference the specific task or issue they address, which proved highly valuable when verifying the updates resolved during the Week 3 audit.

These standards were not enforced through automated checking scripts or pre-commit hooks, but through team reviews at each sprint boundary described in Section 2.2.1. The Week 3 audit sprint served as the project's single most rigorous standards check, since several of the sixteen defects it identified (detailed in Chapter 8) were direct violations of the parameterized-SQL and connection-pooling rules that were introduced under time pressure during earlier weeks and caught only when dedicated review time was allocated.

7 APPLICATION

This chapter describes how the developed platform, named Linkmap, is applied in real-world scenarios to conduct Open-Source Intelligence (OSINT) investigations and analyze narrative patterns on social networks. By combining automated browser collection, structured relational storage, and graph database traversals, the system translates raw social network posts into actionable insights. The primary application domains are detailed below.

7.1 Narrative and Trend Monitoring

A core application of this system is tracking the spread of topics, hashtags, and public communication trends. Traditional research methods require analysts to scroll timelines manually and compile keywords. This platform automates this by extracting hashtags, external web links, and keyword-defined topics from collected post texts.

1. **Hashtag Trends:** Analysts can quickly rank hashtags by frequency and identify co-occurrence patterns (which hashtags appear together in the same posts).
2. **Topic Classification:** By matching keywords against configured maps, the system groups posts into broader discussion clusters (such as "Electoral Politics" or "Media Broadcasts"). This is applied to monitor how different accounts participate in specific narrative discussions.

7.2 Coordinated Activity Detection

The platform plays a key role in identifying coordinated communication campaigns on social networks. Coordinated behavior (where multiple accounts work together to amplify a hashtag or link) is difficult to spot manually because posts are spread across different timelines and times.

1. **Burst Detection:** The system monitors posting frequencies and alerts analysts when an account or group of accounts posts the same hashtag set within a narrow time window (e.g., within a single hour).
2. **Link Sharing Patterns:** By tracking shared external domains, the platform identifies when multiple distinct accounts repeatedly promote the same external URLs, helping analysts detect promotional or coordinated information operations.

7.3 Network and Influence Analysis

Using a graph database (Neo4j) allows the platform to analyze the structure of relationships between users, hashtags, and topics. This has several direct analytical applications:

1. **Key Influencer Identification:** The system calculates central metrics across the graph to find accounts that receive high levels of engagement or mentions.
2. **Bridge Node Detection:** The platform identifies accounts that bridge separate discussion clusters. For example, it can highlight an account that is mentioned

across different topics, serving as a connector between otherwise disconnected groups of users.

- 3. Mention Chain Tracking:** Analysts can run queries to trace how mentions flow between accounts (e.g., Account A mentioning Account B, who mentions Account C), helping map out chains of influence.

7.4 Intelligence Reporting and Sharing

A critical practical application of the platform is generating shareable intelligence assets. Analysts often need to report findings to decision-makers who do not have access to the active dashboard.

- 1. PDF Generation:** The automated report generator synthesizes database engagement statistics (likes, reposts, posting frequencies) and graph relationship data into a single PDF document.
- 2. Data Lineage:** Every finding in the report is traced back to specific database records, providing a clear audit trail (showing the exact post text, timestamp, and account handle) so that stakeholders can verify the source of the intelligence.

Through these applications, the platform serves as a complete workspace for social network OSINT, moving from raw data gathering to relationship mapping, anomaly detection, and final reporting.

8 Limitations and Future Scope

No system built within a fixed eight-week window is complete, and the platform developed in this project, named Linkmap, is no exception. This chapter records the current limitations of this system candidly, and sets out the directions in which it could reasonably be extended—both those already anticipated in the scope boundaries of Section 1.3, and those that emerged only once the system was built and used.

The limitations below are grouped into four categories: platform coverage, analytical depth, deployment and operations, and engineering process. For each category, a corresponding future scope direction is identified, so that this chapter can also serve as a starting roadmap for any future team—whether at BISAG-N or elsewhere—extending this codebase.

Platform Coverage.

The collection engine currently collects data only from X (Twitter), as established in Section 1.3's scope boundary. Public communication and media narratives in India increasingly span Facebook, Instagram, YouTube, and Telegram as well, and a coordinated campaign visible on X may be only one facet of a wider cross-platform narrative invisible to the current system. Future work should extend the Scraper module (described in Section 4.2) with additional platform-specific collectors sharing the same database storage and graph layers, rather than building a parallel pipeline per platform.

Analytical Depth.

Topic classification in the system is currently keyword-based (using the configuration file `config/topic_map.json`) rather than model-based. While this is fast and transparent, it is brittle to phrasings that the configuration's author did not anticipate, and it cannot detect sentiment, sarcasm, or emerging topics not already defined in the keyword map. Campaign and anomaly detection features (described in Chapter 4) similarly rely on threshold and co-occurrence rules rather than trained models. Future work should introduce an optional machine-learning-based topic and sentiment classifier (such as a fine-tuned transformer model) as an additional enrichment stage in the parser module, while retaining the existing keyword map as a clear fallback option.

Deployment and Operations.

As discussed in Section 3.7.3, the platform currently runs as four co-located processes on a single workstation, has no automated deployment pipeline, no containerization, and no multi-analyst authentication or access control. This is adequate for the single-analyst, trusted-workstation usage pattern targeted by this internship, but would not scale to a shared, always-on departmental deployment. Future work should containerize each of the components (such as using Docker), introduce the candidate multi-server deployment model discussed in Section 3.7.3, and add user authentication and role-based access control ahead of any multi-analyst departmental rollout.

Engineering Process.

Beyond these core categories, three further, more specific extensions were identified directly from feedback and observations during this project:

TABLE 12 : SUMMARY OF LIMITATIONS AND RECOMMENDED FUTURE WORK

Category	Current Limitation	Recommended Future Direction
Platform Coverage	Collects data from X (Twitter) only.	Extend Scraper module to support additional social networks sharing the existing database layers.
Analytical Depth	Keyword-based topic tagging and rule-based campaign detection.	Add an optional machine-learning topic/sentiment classifier as an enrichment step in the parser.
Deployment	Single workstation deployment with no containerization or multi-analyst auth.	Containerize backend and database services, adopt a multi-server setup, and implement RBAC.
Engineering Process	Manual, checkpoint-driven testing and no continuous integration pipeline.	Create an automated unit/integration test suite integrated into a continuous delivery workflow.
Scheduling	Analyst-triggered collection runs only.	Set up cron-based scheduled collection runs for continuous monitoring.
Alerting	Analyst must check the Campaigns page manually.	Implement automated email or dashboard alerts when coordinated activity is flagged.

9 Conclusion and References

This chapter closes the report with a summary assessment of what the project achieved relative to the objectives set out in Section 1.4 (Section 9.1), followed by a list of the technical references, documentation sources, and prior-art works consulted throughout the project (Section 9.2).

9.1 Conclusion

This internship set out to design, build, and validate Linkmap, an end-to-end OSINT collection and analysis platform for social network activity on X, over an eight-week development period at BISAG-N. Measured against the eight objectives set out in Section 1.4, the project succeeded in delivering a working anti-ban scraping engine, a dual-database persistence and graph-construction pipeline, a security-hardened backend, a twelve-page analyst dashboard, and an automated, graph-aware PDF intelligence-report generator—all validated against a real twenty-handle pilot dataset that surfaced genuine, human-interpretable OSINT findings. These included a coordinated hashtag campaign, an anomalous external mention pattern, and institutional cross-promotion between affiliated media handles.

Beyond the delivered features themselves, the project's iterative, sprint-based development approach (Chapter 2) proved well-suited to a small team working under a fixed deadline: building the pipeline layer by layer, integrating continuously against real collected data, and dedicating an entire sprint to a security and defect audit (Chapter 8) together produced a system that resolved twenty-two identified issues, none of which remained open by the end of the internship. The dual-database architecture—PostgreSQL for structured querying, Neo4j for relationship traversal—proved to be a sound design choice throughout, in particular because of the graceful-degradation pattern that keeps every PostgreSQL-backed feature usable even when the graph database is temporarily unavailable.

At the same time, this report has been deliberately candid about what remains unfinished: platform coverage limited to X (Twitter), keyword-based rather than model-based topic classification, a single-workstation deployment model, and the absence of an automated test suite (Chapter 8). None of these limitations undermine the core contribution of the project—a working, integrated demonstration that automated collection, dual-database storage, graph-based relationship analysis, and automated reporting can be combined into a single OSINT pipeline—but each represents a concrete, well-understood next step for any team extending this codebase.

In summary, the project met its stated objectives within the constraints of an eight-week internship, produced a reusable and extensible reference implementation for BISAG-N as intended in Section 1.2, and gave the three-member student team genuine, production-adjacent experience across browser automation, relational and graph database design, backend API engineering, real-time web communication, data visualization, and

automated document generation—the full breadth of skills the project was, from the outset, designed to exercise.

9.2 References

The following documentation, technical references, and prior works were consulted during the design and development of the platform:

- [1]. Neo4j, Inc. Neo4j Cypher Manual. <https://neo4j.com/docs/cypher-manual/>
- [2]. Neo4j, Inc. Neo4j Python Driver Manual. <https://neo4j.com/docs/api/python-driver/>
- [3]. FastAPI. FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [4]. Microsoft / Playwright. Playwright for Python Documentation. <https://playwright.dev/python/>
- [5]. PostgreSQL Global Development Group. PostgreSQL Documentation (Arrays, JSON/JSONB Types). <https://www.postgresql.org/docs/>
- [6]. psycopg. psycopg2 Documentation. <https://www.psycopg.org/docs/>
- [7]. D3.js. D3.js Documentation (Force Layout). <https://d3js.org/>
- [8]. ReportLab. ReportLab User Guide. <https://docs.reportlab.com/>
- [9]. Pydantic. Pydantic v2 Documentation. <https://docs.pydantic.dev/>
- [10]. Bazzell, M. Open Source Intelligence Techniques: Resources for Searching.
- [11]. OWASP Foundation. OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/>
- [12]. Robinson, I., Webber, J., and Eifrem, E. Graph Databases (2nd Edition). O'Reilly Media.

In addition to the sources above, this report draws extensively on the project's own internal weekly progress records and codebase documentation compiled by the team between 18th May and 13th July 2026, cited throughout this report as the primary source for architectural and implementation detail.

10 Appendices

This chapter collects supporting reference material that is useful for a reader extending or operating the system, but which would otherwise interrupt the narrative flow of the preceding chapters.

Appendix A: Configuration File Reference

The following outlines the configuration files used to manage the timing, targets, and classification rules of the platform:

TABLE 13 : CONFIGURATION FILE REFERENCE

File	Format	Purpose
config/settings.json	JSON	Manages rate-limiting delays, collection cooldowns, paths, and scrape thresholds.
config/handles.json	JSON	Contains the list of target account handles to collect data from.
config/topic_map.json	JSON	Stores the keyword-to-topic taxonomies used by the parsing pipeline.
.env / backend/.env	ENV	Holds secrets (connection strings, account credentials) and system flags.

Appendix B: Representative API Endpoint List

The following list describes the primary endpoints exposed by the backend server to support the dashboard views:

TABLE 14 : REPRESENTATIVE API ENDPOINT LIST

Method	API Endpoint	Description / Purpose
`GET`	`/api/dashboard/stats`	Fetches overview KPI stats (counts of accounts, posts, relations).
`GET`	`/api/handles`	Returns the directory of tracked accounts.
`GET`	`/api/graph/full`	Returns full Neo4j relationship metrics for network display.
`POST`	`/api/scrapper/start`	Triggers a new data collection thread.
`WS`	`/api/scrapper/ws/{job_id}`	Streams real-time collection logs.
`GET`	`/api/tweet-analysis/tweets`	Supports paginated, multi-filter keyword searches.
`GET`	`/api/data-explorer/export`	Generates a downloadable CSV dataset file.

Method	API Endpoint	Description / Purpose
`GET`	`/api/nexus/graph`	Fetches base network connections for the graph explorer.
`GET`	`/api/nexus/handle/{handle}/details`	Returns profile metadata and node relations for a specific account.
`POST`	`/api/nexus/report`	Triggers the automated generation of the PDF intelligence report.
`GET`	`/health`	Returns the connection status of both PostgreSQL and Neo4j.

Appendix C: Sample Cypher Queries

The following representative Cypher queries illustrate the style of graph queries used throughout the campaign-detection and influence-analysis features described in Chapters 4 and 5:

1. Top shared hashtags between two target accounts:

```
MATCH (a:User)-[r:SHARES_HASHTAG_WITH]-(b:User)
RETURN a.handle, b.handle, r.shared_tag
ORDER BY r.count DESC LIMIT 20
```

2. Direct account-to-account mention network:

```
MATCH (a:User)-[r:MENTIONED_BY_USER]->(b:User)
RETURN a.handle, b.handle, r.count
ORDER BY r.count DESC
```

3. Hashtag co-occurrence burst detection:

```
MATCH (h1:Hashtag), (h2:Hashtag)
WHERE h1 <> h2
WITH h1, h2, count(*) AS co_occurrences
WHERE co_occurrences > threshold
RETURN h1.tag, h2.tag, co_occurrences
```

Report Verification Procedure

Date:

Project Name: OSINT Collection from Social Media Network

Student Name & ID:

1. Sayantan Mandal (26BISAG0507)
2. Shivam Kumar (26BISAG0508)
3. Rishav Pal (26BISAG0509)

	Soft Copy	Hard Copy
Report Format:		
Project Index:		

Sign by Training Coordinator

Sign by Project Guide